ARTIFICIAL INTELLIGENCE

Robust flight navigation out of distribution with liquid neural networks

Makram Chahine†, Ramin Hasani†*, Patrick Kao†, Aaron Ray†, Ryan Shubert, Mathias Lechner, Alexander Amini, Daniela Rus

Copyright © 2023 The Authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original U.S. Government Works

Autonomous robots can learn to perform visual navigation tasks from offline human demonstrations and generalize well to online and unseen scenarios within the same environment they have been trained on. It is challenging for these agents to take a step further and robustly generalize to new environments with drastic scenery changes that they have never encountered. Here, we present a method to create robust flight navigation agents that successfully perform vision-based fly-to-target tasks beyond their training environment under drastic distribution shifts. To this end, we designed an imitation learning framework using liquid neural networks, a brain-inspired class of continuous-time neural models that are causal and adapt to changing conditions. We observed that liquid agents learn to distill the task they are given from visual inputs and drop irrelevant features. Thus, their learned navigation skills transferred to new environments. When compared with several other state-of-the-art deep agents, experiments showed that this level of robustness in decision-making is exclusive to liquid networks, both in their differential equation and closed-form representations.

INTRODUCTION

Intelligence in natural brains differs from today's deep learning systems. The differences are rooted in the ways that these systems learn to make sense of their environment and manipulate it to achieve their goals. Natural learning systems interact with their environments to understand their world. We define understanding as the ability to capture causality and contexts while learning abstract concepts to reason, plan, and control (*1*). This rich representation learning capability allows agents to extrapolate and perform inference and credit assignment even in unseen scenarios and out of distribution (OOD), which is currently beyond the performance achievable with either overparameterized deep learning systems (*2*) or classical statistical learning theory analysis (*3*), because the former overfit to their training data and the latter requires the independent and identically distributed condition on the data distribution.

Studying natural brains effectively narrows the search space of possible algorithms for acquiring intelligent behavior. For instance, neural circuits in brains are much more robust to perturbations and distribution shifts than deep neural networks while also being more flexible in tackling uncertain events compared with deep learning systems. This is because they deploy both unconscious (to facilitate changes in distributions faster) and conscious (to distill causal structure of data and manipulate learned concepts and models) processes (*1*) for decision-making. In contrast, today's deep learning systems are incapable of capturing causality using extracted concepts and features explicitly, despite being able to learn quickly from implicit types of observations.

We find inspiration in the foundational properties of natural learning systems to transform our current deep representation learning frameworks, especially for real-world deployment of artificial learning systems as intelligent agents. In this article, we explore how biologically inspired priors on neural models and network architectures can lead to more flexible, robust, and understandable decision-making for autonomous robots. In particular, we investigate how to construct agents capable of generalizing and achieving zero-shot transfer to new environments for some tasks. To study zero-shot transfer, we focused on end-to-end drone navigation tasks from pixel inputs. For example, consider a scenario where the objective is for a drone agent to navigate to a static target placed in a forest environment. We trained neural network agents on a few runs of data collected by a human pilot in an offline supervised learning setting and observed how the agents transferred under drastic changes in scenery and conditions, such as changing from wild to urban landscapes across multiple seasons. Performing vanilla imitation learning on raw expert demonstrations is sample efficient but generally results in poor closed-loop (active) testing performance due to policy stationarity and compounding errors (*4*).

Specifically, we aimed to develop learning-based solutions to robot flight control that are robust and transferable to novel environments. We studied this problem in the context of the flight hiking task, where a quadrotor robot controller is trained, in one environment, to recognize and navigate toward a target by imitation learning. The resulting policy is then deployed to recognize and move to the target iteratively in different environments. Achieving good out-of-distribution performance requires that trained models learn representations that are causally associated with the task, compositionally resilient, and independent of the environmental context.

There has been extensive research on improving the generalization performance of few-shot, one-shot, and zero-shot imitation learning agents by adopting augmentation strategies (*5*, *6*), human interventions (*7*, *8*), goal conditioning (*9*–*12*), reward conditioning (*13*–*15*), task embedding (*16*), and meta-learning (*17*). These advances help design a better gradient descent–based learning scheme without consideration of the structure of the underlying policy architecture. In contrast, there is evidence that brain-inspired

Massachusetts Institute of Technology (MIT), Cambridge, MA, USA.
*Corresponding author. Email: rhasani@mit.edu
†These authors contributed equally to this work.

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

人工智能

利用液体神经网络实现分布外鲁棒飞行导航

版权所有 © 2023 作者，保留部分权利；独家许可方为美国科学促进会。不对原始美国政府作品主张版权。

Makram Chahine†, Ramin Hasani†*, Patrick Kao†, Aaron Ray†, Ryan Shubert, Mathias Lechner, AlexanderAmini, Daniela Rus

自主机器人能够从离线人类演示中学习执行视觉导航任务，并在其训练过的同一环境中很好地泛化到在线场景和未见场景。对这些智能体而言，要更进一步，并稳健地泛化到从未遇到过、场景发生剧烈变化的新环境，仍具有挑战性。在此，我们提出一种方法，用于构建稳健的飞行导航智能体，使其能够在剧烈分布偏移下，在训练环境之外成功执行基于视觉的飞向目标任务。为此，我们设计了一个使用液体神经网络的模仿学习框架；液体神经网络是一类受大脑启发、具有因果性并能适应变化条件的连续时间神经模型。我们观察到，液体智能体学会从视觉输入中提炼给定任务，并丢弃无关特征。因此，它们学到的导航技能可迁移到新环境。与若干其他最先进的深度智能体相比，实验表明，这种决策稳健性仅存在于液体网络中——无论是在其微分方程表示还是闭式表示中。

引言

自然大脑中的智能不同于当今的深度学习系统。这些差异源于这些系统学习理解其环境并操纵环境以实现目标的方式。自然学习系统与环境交互以理解其世界。我们将理解定义为在学习抽象概念以进行推理、规划和控制的同时，捕捉因果关系与上下文的能力（1）。这种丰富的表征学习能力使智能体即使在前所未见的场景和分布外（OOD）情况下也能进行外推、执行推理和信用分配；目前，这一能力超出了过参数化深度学习系统（2）或经典统计学习理论分析（3）所能达到的性能，因为前者会对其训练数据过拟合，而后者要求数据分布满足独立同分布条件。

研究自然大脑能够有效缩小获取智能行为所需可能算法的搜索空间。例如，与深度神经网络相比，大脑中的神经回路对扰动和分布偏移的鲁棒性要强得多，同时与深度学习系统相比，在处理不确定事件时也更加灵活。这是因为它们部署了无意识过程（用于更快地促使分布发生变化）和有意识过程（用于提炼数据的因果结构并操纵学到的概念和模型）（1）来进行决策。相比之下，当今的深度学习系统尽管能够从隐式类型的观察中快速学习，却无法利用提取出的概念和特征明确捕捉因果关系。

我们从自然学习系统的基本特性中汲取灵感，以改造当前深度表征学习框架，尤其是为了将人工学习系统作为智能体在真实世界中部署。在本文中，我们探讨神经模型和网络架构上的生物启发先验如何为自主机器人带来更灵活、更稳健且更可理解的决策。具体而言，我们研究如何构建能够在某些任务中泛化并实现对新环境零样本迁移的智能体。为了研究零样本迁移，我们聚焦于基于像素输入的端到端无人机导航任务。例如，设想一个场景：目标是让无人机智能体导航至放置在森林环境中的静态目标。我们在离线监督学习设置中，使用人类飞手采集的少量运行数据训练神经网络智能体，并观察这些智能体在场景和条件发生剧烈变化时如何迁移，例如跨多个季节从野外景观变为城市景观。在原始专家演示上执行基础模仿学习具有较高的样本效率，但通常会导致较差的闭环（主动）测试性能，原因在于策略平稳性和误差累积（4）。

具体而言，我们旨在为机器人飞行控制开发基于学习的解决方案，使其对新环境具有鲁棒性且可迁移。我们在飞行徒步任务的背景下研究了这一问题：在该任务中，四旋翼机器人控制器在一个环境中通过模仿学习接受训练，以识别目标并朝目标导航。随后，所得策略被部署到不同环境中，以迭代地识别目标并向目标移动。要实现良好的分布外性能，训练出的模型必须学习与任务因果关联、在组合上具有韧性且独立于环境上下文的表征。

已有大量研究通过采用增强策略(5, 6)、人类干预(7, 8)、目标条件化(9–12)、奖励条件化(13–15)、任务嵌入(16)和元学习(17)，来提升少样本、单样本和零样本模仿学习智能体的泛化性能。这些进展有助于设计更好的基于梯度下降的学习方案，而无需考虑底层策略架构的结构。相比之下，有证据表明，受大脑启发的

麻省理工学院（MIT），美国马萨诸塞州剑桥。*通讯作者。电子邮箱：rhasani@mit.edu
†这些作者对本文贡献相同。

neural dynamics improve the robustness of the decision-making process in autonomous agents, leading to better transferability and generalization in new settings under the same training distribution (*18*–*21*). We aimed to leverage brain-inspired pipelines and empirically demonstrate that if the causal structure of a given task is captured by a neural model from expert data, then the model can perform robustly even OOD. In a set of fly-to-target experiments with different time horizons, we show that a certain class of brain-inspired neural models, namely, liquid neural networks, generalizes well to many OOD settings, achieving performance beyond that of state-of-the-art models.

RESULTS

Our objective is to systematically evaluate how robustly neural architectures can learn a given task from high-dimensional, unstructured, and unlabeled data (for example, no object labels, no bounding boxes, and no constraints on the input space) and transfer their capabilities when we change the environment drastically.

Models that can be trained in one environment to perform in a multitude of other environments have a certain degree of robustness, achieving zero-shot generalization. One approach to obtain such control policies is scaling up their size with structural scaling of neural network policies (*22*–*24*). The universal law of robustness (*25*) suggests that we can obtain robust decision-making systems (worst-case robustness) by training a large neural network model of a size of at least $n \times d$, where n is the number of observations and d is the dimensionality of each sample of the collected dataset. This theoretical result agrees with the recent advances in the impressive capabilities of large models (*26*) on zero-shot and few-shot generalization across a wide variety of tasks, such as natural language processing (*27*–*29*), text-to-image generation (*30*–*32*), scene understanding (*33*), and even vision-based control (*23*, *34*).

On resource-constrained embedded systems such as aerial robots, however, it is infeasible to take advantage of the intriguing generalization properties of large models because of their high computational cost of inference. Recently, a class of compact and brain-inspired continuous-time recurrent neural networks has shown great promise in modeling autonomous navigation of ground (*18*, *19*) and simulated drone vehicles end to end in a closed loop with their environments (*21*). These networks are called liquid time-constant (LTC) networks (*35*), or liquid networks. They are formulated by a nonlinear state-space representation that has the ability to account for external and internal interventions during training (*36*). These properties make them a great candidate for achieving OOD generalization.

It is challenging for machine learning agents to extract the cause and effect of a given task from unstructured and high-dimensional observations. In this work, we created a testbed to assess this capability for various learning systems without any guidance. In particular, the agents must learn/understand the task, the scene, and the object from offline data without any guidance on what the scene, object, and task are. We show how to learn the task and scene end to end from data and how to generalize to unseen and heavily shifted environments with liquid neural networks, where modern alternatives fail.

To this end, we performed a series of quadrotor closed-loop control experiments where pretrained neural architectures were tested for their ability to generalize OOD. Our experiments included the following diverse set of tasks:

1) Fly-to-target tasks. Train on offline expert demonstrations of flying toward a target in the forest and test online in environments with drastic scenery changes.

2) Range test. Take a pretrained network from the fly-to-target task and, without additional training (zero-shot), test how far away we can place the agents to fly toward the target.

3) Stress test. Add perturbations in the image space and measure the success rate of agents under added noise.

4) Attention profile of networks. Apply feature saliency computation to assess the task understanding capabilities of networks via their attention maps.

5) Target rotation and occlusion. Rotate and occlude the target object in the environment and measure whether the agents can complete their flight to the target to measure rotational and occlusion invariance.

6) Hiking with adversaries. Take a pretrained network and use their attention profile for hiking from one object to another one in an environment with a drastic change of scenery, environmental perturbations, and adversarial targets.

7) Triangular (multistep) loop between objects. In a fully new environment, hop from one target to others placed in a triangular state and loop from target to target multiple times.

8) Dynamic target tracking. Take a pretrained network and test how well they can track a moving target in two testing environments with different scenery, lighting, and wind conditions.

We first describe the baseline models we compared against each other. We then present our training setup for the fly-to-target task. Last, we go through our comprehensive testing results in zero-shot closed-loop control.

Baseline models

We compared six different recurrent neural network architectures, namely, long short-term memory (LSTM), gated recurrent unit (GRU)–ordinary differential equation (ODE), ODE–recurrent neural network (RNN), temporal convolutional network (TCN), neural circuit policy (NCP), and closed-form continuous-time (CfC), which are briefly described below. LSTM (*37*) is a discrete-time neural network that consists of a memory cell that is controlled by learnable gates that access, store, clear, and retrieve the information contained in the memory. ODE-RNN (*38*) transforms a vanilla RNN network, for example, an autoregressive fully connected network, into a continuous-time model by using the hidden state of the RNN as the initial state of a neural ODE, for example, an ODE whose derivative is parameterized by a neural network. The time continuity enters the model in the form of the simulation time when solving the ODE with a numerical ODE solver. GRU-ODE pairs a discrete-time GRU (*39*) network with a neural ODE, making it a continuous-time version of a GRU. TCNs (*40*) model the temporal structure of an input sequence by performing a one-dimensional convolution over the time dimension. NCP (*18*) is a sparsely wired four-layer architecture built by LTC cells. The architecture is structurally and semantically inspired by the nervous system of the *Caenorhabditis elegans* worm (*19*). The CfC model (*21*) is a sparsely wired four-layer architecture built by numerical approximation of the closed-form solution of LTCs and is faster to compute than ODE-based models.

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

神经动力学提升了自主智能体决策过程的鲁棒性，从而在相同训练分布下的新设置中带来更好的可迁移性和泛化能力(18–21)。我们旨在利用受大脑启发的流程，并通过实证表明，如果神经模型能够从专家数据中捕捉到给定任务的因果结构，那么该模型即使在分布外(OOD)情况下也能稳健运行。在一组具有不同时间视界的飞向目标实验中，我们表明某一类受大脑启发的神经网络模型，即液体神经网络，能够很好地泛化到许多 OOD 设置，其性能超越了最先进的模型。

结果

我们的目标是系统性地评估神经架构能够以多高的鲁棒性从高维、无结构且无标签的数据（例如，没有目标标签、没有边界框，并且对输入空间没有约束）中学习给定任务，并在我们大幅改变环境时迁移其能力。

能够在一个环境中训练、并在大量其他环境中执行任务的模型具有一定程度的鲁棒性，能够实现零样本泛化。获得此类控制策略的一种方法是通过神经网络策略的结构化扩展来扩大其规模（22–24）。鲁棒性普适定律（25）表明，我们可以通过训练一个规模至少为 $n \times d$ 的大型神经网络模型来获得鲁棒的决策系统（最坏情况鲁棒性），其中 n 是观测值的数量，d 是所采集数据集中每个样本的维度。这一理论结果与近期大模型（26）在多种任务上展现出的令人印象深刻的零样本与少样本泛化能力方面的进展相一致，这些任务如自然语言处理（27–29）、文本到图像生成（30–32）、场景理解（33），甚至基于视觉的控制（23, 34）。

然而，在诸如空中机器人等资源受限的嵌入式系统上，由于大型模型的推理计算成本很高，利用其引人注目的泛化特性是不可行的。最近，一类紧凑且受大脑启发的连续时间递归神经网络展现出巨大潜力，可在与其环境构成的闭环中，端到端地建模地面（18、19）和模拟无人机（21）的自主导航。这些网络被称为液态时间常数（LTC）网络（35），或称液态网络。它们通过一种非线性状态空间表示来构建，该表示能够在训练期间考虑外部和内部干预（36）。这些特性使它们成为实现分布外（OOD）泛化的理想候选。

机器学习智能体要从非结构化且高维的观测中提取给定任务的因果关系是具有挑战性的。在这项工作中，我们构建了一个测试平台，用于在没有任何指导的情况下评估各种学习系统的这一能力。具体而言，智能体必须从离线数据中学习/理解任务、场景和对象，且没有任何关于场景、对象和任务是什么的指导。我们展示了如何从数据中端到端地学习任务和场景，以及如何利用液态神经网络泛化到未见且发生严重偏移的环境，而现代替代方案在这些情况下会失败。

为此，我们进行了一系列四旋翼飞行器闭环控制实验，在实验中，预训练的神经网络架构接受了对其 OOD 泛化能力的测试。我们的实验包括以下多样化的任务集：

1) 飞向目标任务。使用在森林中飞向目标的离线专家演示进行训练，并在场景发生剧烈变化的环境中进行在线测试。

2) 距离范围测试。取一个来自飞向目标任务的预训练网络，在不进行额外训练（零样本）的情况下，测试我们能将智能体放在离目标多远的位置让其飞向目标。

3) 压力测试。在图像空间中添加扰动，并测量智能体在添加噪声下的成功率。

4) 网络的注意力分布。应用特征显著性计算，通过网络的注意力图评估其任务理解能力。

5) 目标旋转与遮挡。在环境中旋转和遮挡目标物体，并测量智能体能否完成飞向目标的飞行，以评估旋转不变性和遮挡不变性。

6) 带对抗的徒步。取一个预训练网络，并利用其注意力分布，在场景剧烈变化、存在环境扰动和对抗性目标的环境中，从一个物体徒步前往另一个物体。

7) 物体之间的三角形（多步）循环。在全新环境中，从一个目标跳转至按三角形状态布置的其他目标，并在目标之间多次循环。

8) 动态目标跟踪。采用预训练网络，并测试其在两个具有不同场景、光照和风况的测试环境中跟踪移动目标的效果。

我们首先介绍参与相互比较的基线模型。随后展示飞向目标任务的训练设置。最后，我们梳理零样本闭环控制下的全面测试结果。

基线模型

我们比较了六种不同的循环神经网络架构，即长短期记忆（LSTM）、门控循环单元（GRU）—常微分方程（ODE）、ODE—循环神经网络（RNN）、时间卷积网络（TCN）、神经电路策略（NCP）以及闭式连续时间（CfC），下面简要介绍这些架构。LSTM（37）是一种离散时间神经网络，由一个记忆单元组成，该记忆单元由可学习门控制，这些门访问、存储、清除和检索记忆中所包含的信息。ODE-RNN（38）通过将RNN的隐藏状态用作神经ODE（例如，其导数由神经网络参数化的ODE）的初始状态，把普通RNN网络（例如自回归全连接网络）转换为连续时间模型。当使用数值ODE求解器求解ODE时，时间连续性以仿真时间的形式进入模型。GRU-ODE将离散时间GRU（39）网络与神经ODE配对，使其成为GRU的连续时间版本。TCN（40）通过对时间维度执行一维卷积来建模输入序列的时间结构。NCP（18）是一种由LTC单元构建的稀疏连接四层架构。该架构在结构和语义上受到秀丽隐杆线虫（Caenorhabditis elegans）神经系统（19）的启发。CfC模型（21）是一种稀疏连接的四层架构，通过LTC闭式解的数值近似构建，其计算速度快于基于ODE的模型。

Fly-to-target task training

The fly-to-target task consists of autonomously identifying a target of interest and performing the flight controls driving the quadrotor toward it using the onboard stabilized camera's sequences of RGB (red, green and blue) images as the sole input (Fig. 1). We initially started the quadrotor about 10 m away from the target and required that the policy guide the drone to within 2 m of the target, with the target centered in the camera frame. The objective was to learn to complete this task entirely from an offline dataset of expert demonstrations. The learned policies were then tested online in a closed loop within the training distribution and in drastically distinct settings. This experimental protocol allows for the principled assessment of performance and generalization capabilities of liquid networks compared with modern deep models (*38*, *41*, *42*).

Closed-loop (online) testing

All seven tasks were evaluated in closed loop with environments by running the neural networks onboard the drone. In the following, we describe our results.

Fly-to-target and zero-shot generalization

In this setting, the quadrotor was placed at a fixed distance from a target of interest seen in the training data. In our evaluation experiments, we used a red camping chair as the target for collision avoidance purposes; because of its relatively large size, it could occupy an important portion of the camera frame without the drone having to get dangerously close. The target was present in the initial field of view, and its position in the initial input frame was randomized but balanced over all runs (each network was tested on the same equal number of right, center, and left starting positions at slightly different altitudes). The initial testing protocol was repeated for a number of testing environments to evaluate the networks' performance in OOD settings but maintained a task structure identical to the one learned from training data. We randomly positioned the quadrotor at a distance of about 10 m from the target, with the latter in the field of view of the onboard camera. We launched the closed-loop policy and observed whether the network could successfully guide the drone to the target. The test was repeated 40 times for each network and in each environment. Success was accounted for when the network was able to both stabilize the drone in a radius of 2 m and maintain the target in the center of the frame for 10 s. Failure cases were identified when the network generated commands that led to an exit of the target from the range of view without the possibility of recovery. We also included cases where the drone failed to reach the target in less than 30 s to account for rare runs where the network generated commands indefinitely, maintaining the drone in a stable position with the target in sight but with no intention of flying toward it.

We tested the policies in four environments we call Training Woods, Alternative Woods, Urban Lawn, and Urban Patio (Fig. 1, B and D). The first corresponded to the target in the woods, respecting roughly the same position and orientation distribution as that of the training set. We thus evaluated the performance of the networks on image sequences from the scenery used during training with different lighting conditions and wind profiles. Alternatively, we moved the chair to a different spot with a different background in a neighboring part of the woods. This experiment evaluated the networks' generalization performance on input data with scenery distribution shifts but maintained proximity to sequences seen during training. To truly evaluate robust task understanding, we subsequently set up the zero-shot domain transfer experiments in a drastically dissimilar environment: a piece of lawn on CSAIL MIT's campus. Although containing green grass and a few trees, the

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

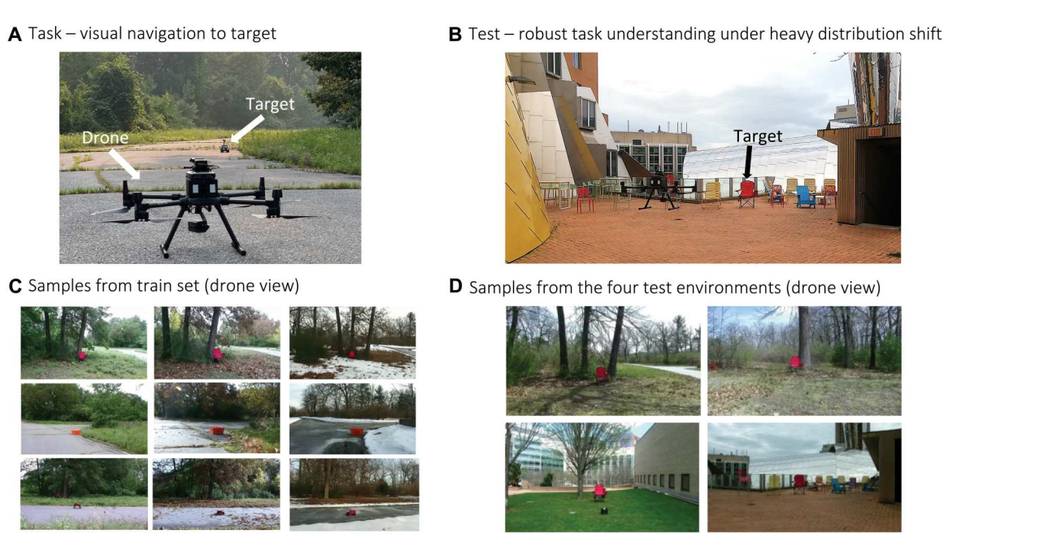


Fig. 1. Sample frames of training and test environments. (**A**) Third-person view of the quadrotor and one of the targets in the Training Woods where data were collected. (**B**) Third-person view of the quadrotor during testing on the Urban Patio, with multiple adversary objects dispersed around the target camping chair. (**C**) Training data frame samples from the quadrotor onboard camera containing various targets (camping chair, storage box, and RC car from top to bottom) and taken during different seasons (summer, fall, and winter from left to right). (**D**) Test data frame samples from the quadrotor onboard camera against each of the four test backgrounds: Training Woods (top left), Alternative Woods (top right), Urban Lawn (bottom left), and Urban Patio (bottom right).

飞向目标任务训练

飞向目标任务包括自主识别感兴趣的目标，并以机载稳定相机拍摄的RGB（红、绿、蓝）图像序列作为唯一输入，执行飞行控制以驱动四旋翼飞行器飞向该目标（图1）。我们最初让四旋翼飞行器从距离目标约10 m处起飞，并要求策略引导无人机到达距目标2 m以内，同时使目标位于相机画面中央。目标是仅利用专家演示的离线数据集学习完成该任务。随后，学习到的策略在训练分布内以及差异显著的环境中，以闭环方式在线测试。该实验方案能够对液态网络与现代深度模型（38, 41, 42）相比的性能和泛化能力进行原则性评估。

闭环（在线）测试

所有七项任务均通过在无人机上运行神经网络，在闭环中与环境交互进行评估。下文将描述我们的结果。

飞向目标与零样本泛化

在此设置中，四旋翼无人机被放置在距训练数据中可见的感兴趣目标固定距离的位置。在我们的评估实验中，我们使用红色露营椅作为用于避碰目的的目标；由于其尺寸相对较大，它可以在无人机无需危险地靠近的情况下占据相机画面的重要部分。目标出现在初始视野中，并且其在初始输入帧中的位置在所有运行中随机化但保持均衡（每个网络都在略有不同的高度上，以相同数量的右、中、左起始位置进行了测试）。初始测试方案在多个测试环境中重复进行，以评估网络在OOD 设置中，但保持了与从训练数据中学到的任务结构相同的任务结构。我们将四旋翼无人机随机放置在距目标约 10 m 的位置，并使目标位于机载相机的视野中。我们启动闭环策略，并观察网络能否成功引导无人机到达目标。该测试针对每个网络、在每个环境中重复 40 次。当网络能够将无人机稳定在 2 m 半径内，并将目标保持在画面中心 10 s 时，即判定为成功。当网络生成的指令导致目标离开视野范围且无法恢复时，即判定为失败案例。我们还纳入了无人机未能在 30 s 内到达目标的情况，以涵盖少数运行中网络无限生成指令、使无人机保持稳定位置且目标在视野内但无意飞向目标的情形。

我们在四个环境中测试了这些策略，分别称为训练树林、替代树林、城市草坪和城市露台（图1B和D）。第一个环境对应于树林中的目标，大致保持与训练集相同的位置和朝向分布。因此，我们评估了网络在训练时所用场景的图像序列上的性能，且这些图像序列具有不同的光照条件和风况。或者，我们将椅子移到树林邻近区域中一个背景不同的位置。该实验评估了网络在具有景物分布偏移、但仍与训练期间所见序列保持接近的输入数据上的泛化性能。为了真正评估稳健的任务理解，我们随后在一个截然不同的环境中进行了零样本域迁移实验：麻省理工学院CSAIL校区的一片草坪。尽管这里也有绿草和少量树木，但

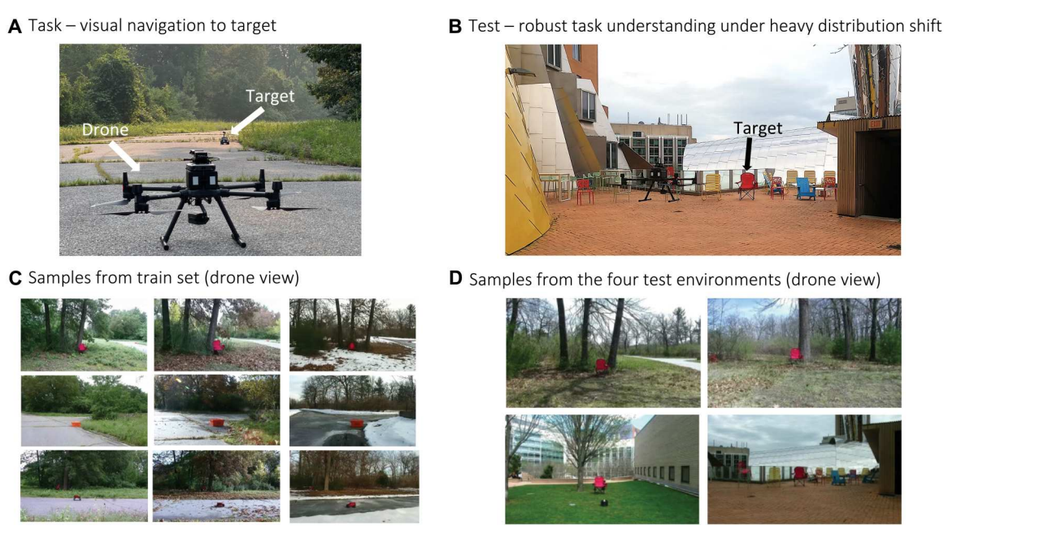


图1. 训练和测试环境的示例帧。（A）训练树林中四旋翼无人机和一个目标的第三人称视角，数据在此处采集。（B）在城市露台测试期间四旋翼无人机的第三人称视角，多个对抗物体分散在目标露营椅周围。（C）来自四旋翼无人机机载相机的训练数据帧样本，包含各种目标（从上到下依次为露营椅、储物箱和遥控车），并拍摄于不同季节（从左到右依次为夏季、秋季和冬季）。（D）来自四旋翼无人机机载相机的测试数据帧样本，分别对应四种测试背景：训练树林（左上）、替代树林（右上）、城市草坪（左下）和城市露台（右下）。

landscape was largely different from the woods, as shown in Fig. 1 (A to D). Frames contained buildings, windows, large reflective metallic structures, and the artificial contrast from the geometric shades they induced. Lighting conditions at different times of day and varying wind levels added additional perturbations.

We lastly examined the networks′ generalization capabilities in an OOD environment consisting of a brick patio. In this environment, the background, including a number of man-made structures of different shapes, colors, and reflectivity, drastically differed from the training environment. Moreover, we added an extra layer of complexity to this experiment by positioning a number of other chairs in the frame of different colors (including red) and sizes. This ultimate test, including real-world adversaries, required robustness in the face of extremely heavy distribution shifts in addition to proper task understanding to recognize and navigate toward the correct target. Table 1 shows the success rates of the different network architectures across the four environments. Both variants of liquid networks outperformed other models in terms of task completion. Moreover, we observed that, generally, recurrent networks worked better when tested within the same training distribution, such as the Training Woods and Alternative Woods environment. Meanwhile, in OOD scenarios, their performance dropped considerably. However, in such OOD settings, liquid networks stood out, with a sparse liquid CfC network having a success rate of 90% in Urban Lawn and 67.5% in Urban Patio (which contains more natural adversaries).

Range tests

Beyond the extensive testing from the fixed distance of 10 m from the target (which was roughly the starting position in the training data), we also performed a range test to explore how the networks react to an extension of the task unseen in the training data. The experimental scenery was that of the Training Woods setup presented above but with the drone′s starting positions growing in distance to the target, thus requiring an increasingly acute task understanding and perception capacity to successfully achieve the objective. We ran 10 experiments for each of 10-, 20-, and 30-m distance from the target. At the time of testing, the target lay in the shade, making it hard to detect against the brighter components in the image, such as tarmac, sky, and trees. The performance of each of the networks is summarized in Table 2.

The results of this experiment show strong evidence that liquid networks have the ability to learn a robust representation of the task they are given and can generalize well to OOD scenarios where other models fail. This observation is aligned with recent works (*21*) that showed that liquid networks are dynamic causal models (DCMs) (*43*) and can learn robust representations for their perception modules to perform robust decision-making. In particular, CfC networks managed to fly the drone autonomously from twice and three times the training distance to their targets from raw visual data with a success rate of 90 and 20%, respectively. This is in contrast to an LSTM network that lost the target in every single attempt at both these distances, leading to a 0% success rate. Only ODE-RNN managed to achieve a single success at 20 m among the nonliquid networks, and none reached the target from 30 m.

Stress tests

In addition to the flight experiments with the networks running online, we performed a series of offline stress tests as another measure of the robustness of different neural architectures to distribution shifts by perturbing the input frames to the network and observing changes in network outputs. Ideally, networks should maintain high performance despite varied environmental conditions. To simulate this domain randomization, we perturbed images from collected sequences, changing brightness, contrast, saturation, and noise to mimic lighting differences, color variations, and sensor noise found in real-world testing. We then measured whether networks substantially changed their control outputs, indicating failure to generalize to new environments, or generated consistent outputs across different perturbations, displaying resilience to distribution shifts.

To quantify the difference in network outputs, we applied a perturbation to the frames of a recorded sequence, fed the original and perturbed frames to the network, and recorded the original and perturbed output velocities generated by the network. We then applied Forward Euler numerical integration to integrate the velocities into position trajectories. We thus measured the final distance between the last point on the two trajectories, recorded this as the perturbed distance, and repeated this process for different perturbation types and amplitudes. Figure 2 shows the results of stress testing for noise, brightness, contrast, and saturation perturbations.

In all four cases, CfCs demonstrated superior robustness compared with their counterparts. They deviated the least from their nominal trajectories. This observation is consistent with our

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

Table 1. Closed-loop evaluation of the fly-to-target task for the trained policies in four testing environments. Quantitative results are success rates. Higher is better. (*N* = 40)

Algorithm	Environment			
	Training Woods	Alt. Woods	Urban Lawn	Urban Patio
LSTM	82.5%	92.5%	62.5%	27.5%
GRU-ODE	50%	17.5%	32.5%	17.5%
ODE-RNN	62.5%	82.5%	17.5%	25%
TCN	7.5%	0%	0%	0%
NCP (ours)	100%	95%	57.5%	52.5%
CfC (ours)	85%	95%	90%	67.5%

Table 2. Range test evaluation of the trained policies in the Training Woods environment. Quantitative results are success rates. Higher is better. (*N* = 10)

Algorithm	Initial distance (m)		
	10	20	30
LSTM	100%	0%	0%
GRU-ODE	70%	0%	0%
ODE-RNN	80%	10%	0%
TCN	5%	0%	0%
NCP (ours)	100%	50%	10%
CfC (ours)	100%	90%	20%

景观与树林有很大不同，如图1（A至D）所示。帧中包含建筑物、窗户、大型反光金属结构，以及它们所产生的几何阴影带来的人工对比度。一天中不同时段的光照条件和不同的风力水平又增加了额外的扰动。

最后，我们考察了网络在一个由砖砌露台构成的 OOD 环境中的泛化能力。在该环境中，包括若干形状、颜色和反射率各异的人造结构在内的背景，与训练环境截然不同。此外，我们还通过在该场景画面中放置若干其他颜色（包括红色）和尺寸的椅子，为这项实验增加了一层复杂性。这项包含真实世界对抗因素的终极测试，不仅要求具备恰当的任务理解能力，以识别并朝正确目标导航，还要求在面对极其严重的分布偏移时保持鲁棒性。表 1 展示了不同网络架构在四种环境中的成功率。两种液态网络变体在任务完成方面均优于其他模型。此外，我们观察到，总体而言，循环网络在同一训练分布内测试时表现更好，例如训练树林和替代树林环境。与此同时，在 OOD 场景中，它们的性能大幅下降。然而，在此类 OOD 设置中，液态网络表现突出，其中稀疏液态 CfC 网络在城市草坪中的成功率为 90%，在城市露台中为 67.5%（后者包含更多自然对抗因素）。

范围测试

除了在距目标固定10 m（大致为训练数据中的起始位置）进行的大量测试外，我们还进行了距离测试，以探究网络在训练数据中未出现过的任务扩展情形下会如何反应。实验场景采用上文所述的 Training Woods 设置，但无人机起始位置与目标的距离逐渐增大，因此需要越来越敏锐的任务理解能力和感知能力才能成功达成目标。我们分别对距目标10 m、20 m和30 m的情况各进行了10次实验。测试时，目标位于阴影中，因而难以在柏油路面、天空和树木等图像中较亮的部分背景下被检测到。各网络的性能总结于表2。

本实验结果表明，有强有力的证据显示，液态网络能够学习到其所获任务的稳健表征，并能很好地泛化到其他模型失败的 OOD 场景。这一观察与近期研究（21）一致，该研究表明液态网络是动态因果模型（DCMs）（43），能够为其感知模块学习稳健表征，从而实现稳健决策。具体而言，CfC 网络成功仅凭原始视觉数据，让无人机自主从训练距离的两倍和三倍处飞抵目标，成功率分别为90%和20%。相比之下，LSTM 网络在这两种距离下的每一次尝试中都丢失了目标，导致成功率为0%。在非液态网络中，只有ODE-RNN 在20 m处实现了一次成功，而从30 m处出发的网络均未到达目标。

压力测试

除了让网络在线运行的飞行实验外，我们还进行了一系列离线压力测试，作为衡量不同神经架构对分布偏移鲁棒性的另一项指标：通过扰动输入到网络的帧并观察网络输出的变化来进行评估。理想情况下，网络应能在各种环境条件下保持高性能。为了模拟这种域随机化，我们对采集序列中的图像进行扰动，改变亮度、对比度、饱和度和噪声，以模拟真实世界测试中遇到的照明差异、颜色变化和传感器噪声。随后，我们测量网络是否会大幅改变其控制输出（这表明其无法泛化到新环境），还是会在不同扰动下生成一致的输出（从而表现出对分布偏移的韧性）。

为了量化网络输出的差异，我们对一段录制序列的帧施加扰动，将原始帧和扰动帧输入网络，并记录网络生成的原始输出速度和扰动输出速度。然后，我们采用前向欧拉数值积分将这些速度积分成位置轨迹。由此，我们测量了两条轨迹上最后一个点之间的最终距离，并将其记录为扰动距离；针对不同的扰动类型和幅度，我们重复了这一过程。图2展示了针对噪声、亮度、对比度和饱和度扰动的压力测试结果。

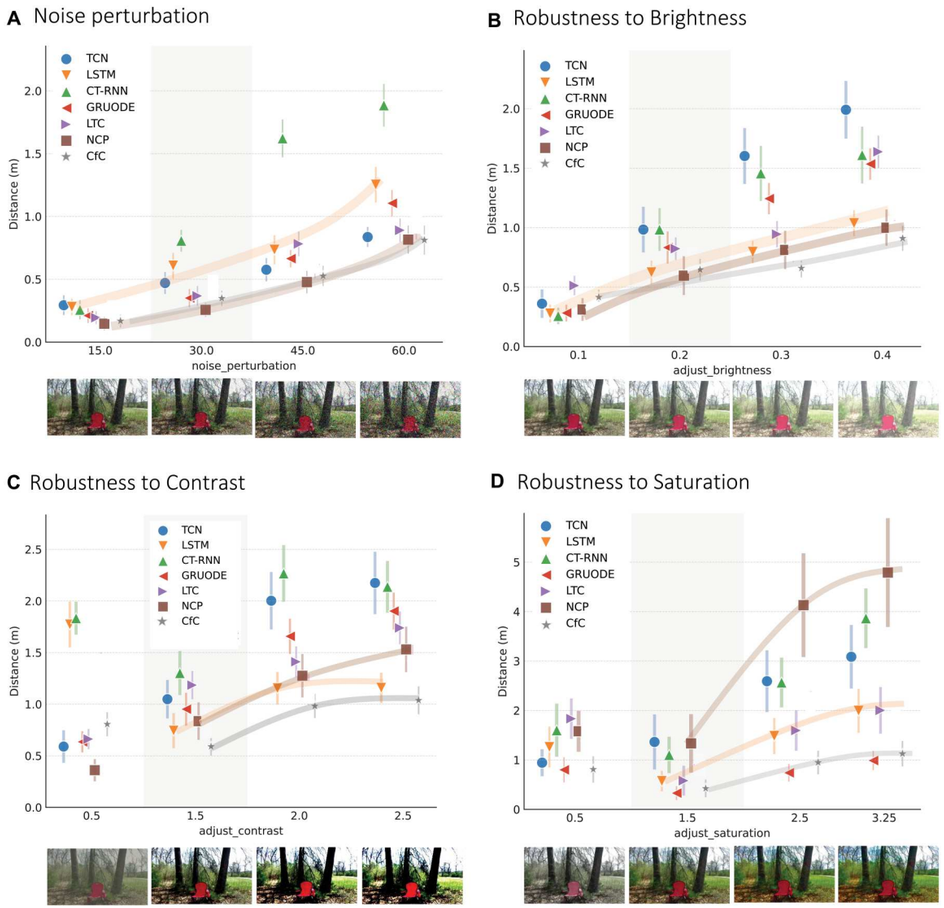
在所有四种情况下，CfCs 与对应模型相比均表现出更优的鲁棒性。它们偏离名义轨迹的程度最小。这一观察结果与我们的

表1. 训练后的策略在四种测试环境中执行飞向目标任务的闭环评估。定量结果为成功率。越高越好。（N = 40）

算法	环境			
	训练林地	Alt. 林地	城市 Lawn	城市露台
LSTM	82.5%	92.5%	62.5%	27.5%
GRU-ODE	50%	17.5%	32.5%	17.5%
ODE-RNN	62.5%	82.5%	17.5%	25%
TCN	7.5%	0%	0%	0%
NCP（我们的方法）	100%	95%	57.5%	52.5%
CfC（本文方法）	85%	95%	90%	67.5%

表 2. 在 Training Woods 环境中对训练后的策略进行范围测试评估。定量结果为成功率。越高越好。（N = 10）

算法	初始距离（m）		
	10	20	30
LSTM	100%	0%	0%
GRU-ODE	70%	0%	0%
ODE-RNN	80%	10%	0%
TCN	5%	0%	0%
NCP（本文）	100%	50%	10%
CfC（本文）	100%	90%	20%



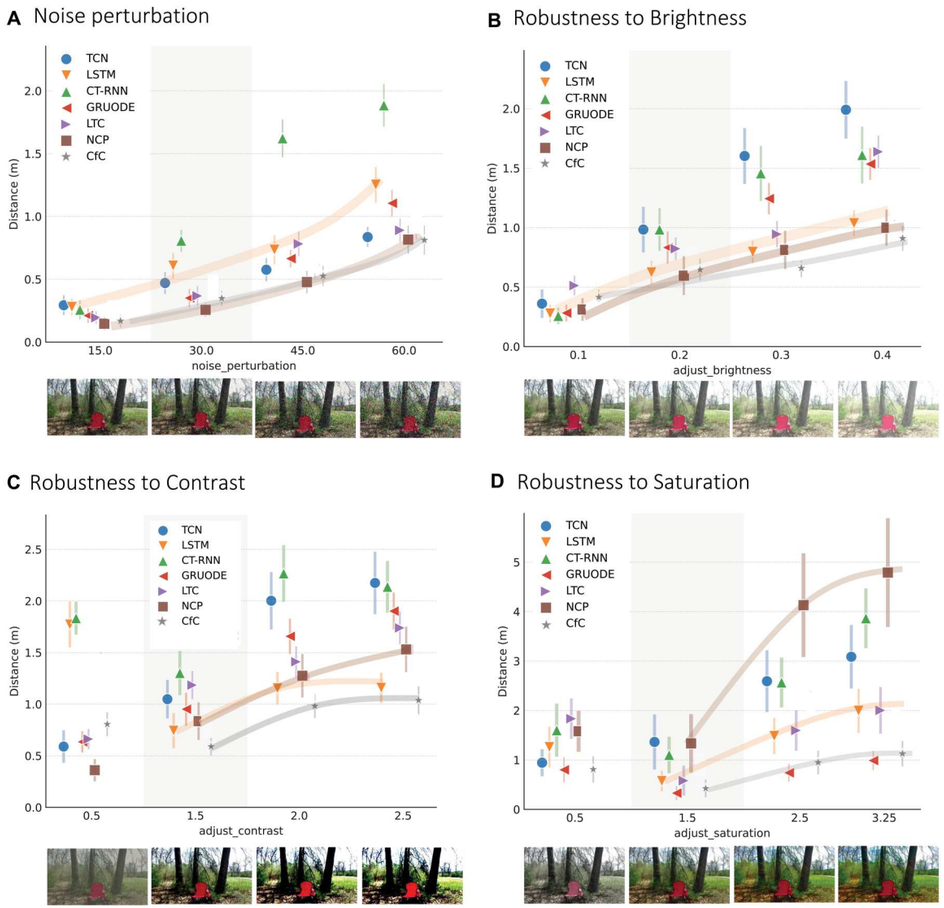


Fig. 2. Noise, brightness, contrast, and saturation stress tests. These experiments run both an original image sequence collected by the drone and a perturbed version through a network, integrate the output velocity commands, and record the final displacement between sequences (lower is better). Bars show the SD across 10 runs. Trendlines for NCP, LSTM, and CFC are shown with shaded brown, orange, and gray lines, respectively (N = 10). (**A**) Distances when Gaussian noise with SD x is added to image frames. (**B**) Distances when brightness is shifted by x times the maximum pixel value. (**C**) Distances when the contrast in the image is multiplied by x. Note that the first data point, 0.5, is actually a reduction in contrast and moves all pixel values toward the mean, whereas the other three data points increase contrast and move pixel values away from the mean. (**D**) Distances when saturation (S in HSV colorspace) is multiplied by x. Note the first data point, 0.5, reduces the saturation of colors in the image, whereas the other three data points increase saturation.

图 2. 噪声、亮度、对比度和饱和度压力测试。这些实验将由无人机采集的原始图像序列以及扰动版本分别输入网络，对输出的速度指令进行积分，并记录序列之间的最终位移（越低越好）。条形表示 10 次运行的标准差。NCP、LSTM 和 CFC 的趋势线分别以棕色、橙色和灰色阴影线表示（N = 10）。(A) 向图像帧中加入标准差为 x 的高斯噪声时的距离。(B) 亮度偏移为最大像素值 x 倍时的距离。(C) 图像对比度乘以 x 时的距离。注意，第一个数据点 0.5 实际上是降低对比度，并使所有像素值向均值靠拢；而其他三个数据点则提高对比度，使像素值远离均值。(D) 饱和度（HSV 色彩空间中的 S）乘以 x 时的距离。注意，第一个数据点 0.5 会降低图像颜色的饱和度，而其他三个数据点则提高饱和度。

active testing results, where in OOD scenarios, CfCs stood out in task completion success rate. Liquid NCPs also showed great resiliency to noise, brightness, and contrast perturbations because their performance was hindered by changes in input pixels′ saturation level. In this offline setting, LSTMs were also among the most robust networks after liquid networks. However, their OOD generalization was poor compared with liquid networks.

Attention profile of networks

To assess the task understanding capabilities of all networks, we computed the attention maps of the networks via the feature saliency computation method called VisualBackProp (*44*) to the CNN backbone that precedes each recurrent network. VisualBackProp associates importance scores to input features during decision-making. The method has shown promise in real-world robotics applications where visual inputs are processed by convolutional filters first. The attention maps corresponding to the convolutional layers of each network in a test case scenario are shown in Fig. 3. We observed that the saliency maps of liquid networks (both NCPs and CfCs) were much more sensitive to the target from the start of the flight compared with those of other methods.

We show more comprehensive saliency maps in different scenarios in figs. S1 to S3. We observed that the quality of the attention

主动测试结果一致：在 OOD 场景中，CfCs 在任务完成成功率方面表现突出。Liquid NCPs 对噪声、亮度和对比度扰动也表现出很强的韧性，因为其性能会受到输入像素饱和度水平变化的影响。在此离线设置中，LSTMs 也是仅次于液态网络的最鲁棒网络之一。然而，与液态网络相比，它们的 OOD 泛化能力较差。

网络的注意力分布

为了评估所有网络的任务理解能力，我们采用名为 VisualBackProp (44) 的特征显著性计算方法，计算了各网络在位于每个循环网络之前的 CNN 骨干网络上的注意力图。VisualBackProp 在决策过程中将重要性分数与输入特征相关联。该方法在视觉输入首先由卷积滤波器处理的真实世界机器人应用中已显示出潜力。在测试用例场景中，各网络卷积层对应的注意力图如图 3 所示。我们观察到，与其他方法相比，液体网络（NCPs 和 CfCs）的显著性图从飞行一开始就对目标更为敏感。

我们在图 S1 至 S3 中展示了不同场景下更全面的显著性图。我们观察到，注意力

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

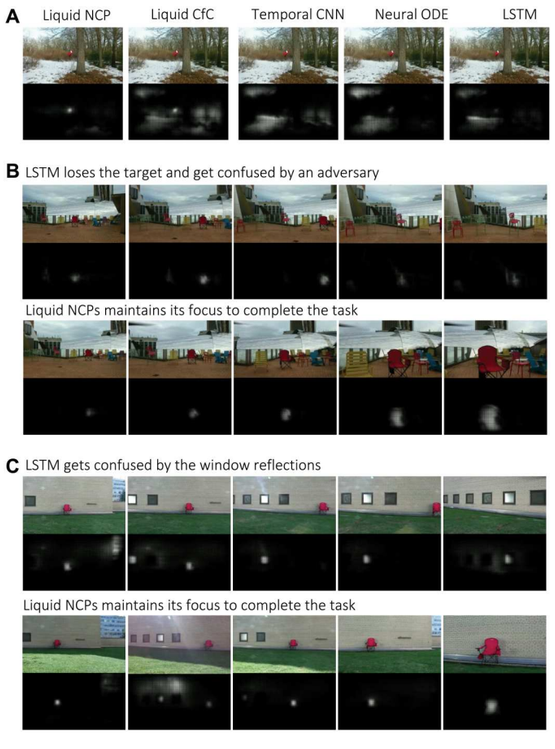


Fig. 3. Evolution of flight-time networks′ attention maps. (**A**) A frame illustration of the attention profile of different networks computed by the VisualBackProp saliency map computation algorithm (*44*). The top row shows the input image frame. The bottom row shows the saliency map. The brighter regions indicate a greater concentration of the convolutional layer′s attention. Saliency scores range between 0 (black) and 1 (white). (**B**) A flight case study where an LSTM agent loses its target and gets confused by an adversary. Left to right shows the progress time. (**C**) A similar scenario as in (B), in a different OOD testing environment.

maps of the convolutional layers drastically improved for all networks when data augmentation was used during training.

The attention maps are greatly advantageous because they allow us to inspect and interpret reasons for a failed or successful decision-making process by a network under distribution shifts. For instance, we illustrate two OOD test cases in Fig. 3 (B and C), in which LSTM agents lost their target because of environmental adversaries, whereas, in very similar cases, liquid networks managed to keep their focus on the target and complete their tasks. We observed that learning more robust perception module filters (convolutional filters) can enhance the generalization of autonomous flight agents. This is why the robustness of the transformation from convolutional heads to the recurrent module (the decision-making compartment) highly depends on the choice of model. Our observations suggest that liquid networks improve representation learning on both of these fronts compared with other modern architectures.

Target′s rotation and occlusion

We also conducted a series of experiments pertaining to the appearance of the target. More specifically, we evaluated the trained networks on the fly-to-target task in the Urban Lawn setting from the nominal distance of 10 m but first rotated and then partially occluded the target (with a black curtain, vertically along the middle). These experiments tested the robustness and invariance of the learned task representation that such alterations of the target can have on different architectures. The results for each of the networks are presented in Table S5.

We analyzed the results of these experiments relative to the third column of Table 1 to understand the effect of our modifications on performance. In the case of rotation, GRU-ODE is the architecture that suffered the most, managing only one success between both the 90° and 180° tests (2.5%), although it reached the upright target roughly a third of the time (32.5%) in the same environment. At both angles, LSTM saw its success rates halved compared with the upright scenario. Both NCP and ODE-RNN had their success rates reduced by around 40% on the 90° scenario, whereas CfC only lost 10%. In the 180° case, the latter three networks all maintained performance levels close to those in the initial upright setting. With the target occluded, only ODE-RNN and NCP managed to maintain their nominal performance (the latter even slightly improving on it). GRU-ODE, LSTM, and CfC saw performance reductions of 70, 60, and 30%, respectively.

Zero-shot hiking with adversaries

In this task, the target chairs were placed in the environment in the disposition depicted in fig. S9. The test was conducted under extremely bright sunlight, adding substantial perturbation to multiple network architectures′ performance, with intense glare reflecting off of the hike targets. Moreover, we placed adversarial objects scattered in the scene (including a blue bin, a yellow chair, and two red chairs of different materials and builds). This setup required robustness in the face of extremely heavy distribution shifts in addition to proper task understanding to recognize and navigate toward the correct target at each step. The test was conducted on a cloudy day, with the target chairs′ colors true to those appearing in the training data.

Table S6 contains the success rate results for this test. TCN and GRU-ODE models severely struggled to detect targets in this setup (less than 45% of runs reached the first checkpoint and none attended to the second). Liquid networks performed best in this scenario, with a CfC completing 14 of 20 runs of the hiking task with a target horizon of three. The results of this experiment show strong evidence that liquid networks have the ability to learn a robust representation of the task they are given and can generalize well to OOD scenarios where other models fail or perform poorly (see figs. S7 and S8 for examples of the perturbations and adversaries perceived by the drone′s input camera). This observation is aligned with recent works (*21*) that showed that liquid networks are DCMs (*43*) and can learn robust representations for their perception modules to perform robust decision-making.

Triangular loop between objects

In this task, we demonstrated that our approach can generalize to an infinite number of hiking steps by setting up the three chairs as vertices of an equilateral triangle of sides equal to 10 m, as illustrated in Fig. 4. We programmed the quadrotor to turn an angle of 120° to the right after detection of a checkpoint (instead of flying up like in the experiments described above) to increase the likelihood of the next target appearing in the field of view. Using the CfC architecture, we

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

图3. 飞行时间网络注意力图的演化。（A）由 VisualBackProp 显著性图计算算法（44）计算得到的不同网络注意力分布的帧示意图。顶行显示输入图像帧。底行显示显著性图。较亮区域表示卷积层注意力更集中。显著性分数范围为0（黑色）到1（白色）。（B）一个飞行案例研究，其中 LSTM 智能体丢失目标并被对手迷惑。从左到右表示时间进程。（C）与（B）类似的情景，但处于不同的 OOD 测试环境。

在训练期间使用数据增强时，所有网络的卷积层注意力图都得到了显著改善。

注意力图具有很大优势，因为它们使我们能够检查并解释在分布偏移下网络决策过程失败或成功的原因。例如，我们在图3（B和C）中展示了两个OOD测试案例；在这些案例中，LSTM智能体因环境对抗因素而丢失目标，而在非常相似的情况下，液态网络成功地将注意力保持在目标上并完成了任务。我们观察到，学习更稳健的感知模块滤波器（卷积滤波器）可以增强自主飞行智能体的泛化能力。这就是为什么从卷积头到循环模块（决策部分）的变换的鲁棒性高度取决于模型选择。我们的观察表明，与其他现代架构相比，液态网络在这两个方面都改善了表征学习。

目标的旋转与遮挡

我们还针对目标外观开展了一系列实验。更具体地说，我们在Urban Lawn场景中，从10 m的标称距离出发，评估了训练好的网络在飞向目标任务上的表现，但先旋转目标，然后对目标进行部分遮挡（用黑色幕布沿垂直中线遮挡）。这些实验检验了目标发生此类改变时，不同架构所学任务表征的鲁棒性和不变性。各网络的结果见表S5。

我们结合表1第三列分析了这些实验的结果，以了解我们的修改对性能的影响。在旋转情况下，GRU-ODE 是受影响最大的架构，在 90° 和 180° 两项测试中总共仅成功一次（2.5%），尽管在同一环境中它约有三分之一的时间（32.5%）能够到达直立目标。在这两个角度下，LSTM 的成功率相比直立场景均减半。在 90° 场景中，NCP 和 ODE-RNN 的成功率均下降约 40%，而 CfC 仅下降 10%。在 180° 情况下，后三种网络均保持了与初始直立设置相近的性能水平。当目标被遮挡时，只有 ODE-RNN 和 NCP 设法维持了其标称性能（后者甚至略有提升）。GRU-ODE、LSTM 和 CfC 的性能分别下降 70%、60% 和 30%。

对抗环境下的零样本徒步

在本任务中，目标椅子按图 S9 所示的位置布置在环境中。测试是在极其明亮的阳光下进行的，这给多种网络架构的性能带来了显著扰动，强烈的眩光从徒步目标上反射出来。此外，我们还在场景中散布了对抗对象（包括一个蓝色垃圾桶、一把黄色椅子，以及两把材质和结构不同的红色椅子）。这种设置不仅要求模型在每一步都能正确理解任务，以识别并朝正确目标导航，还要求其在面对极重的分布偏移时具备鲁棒性。该测试是在阴天进行的，目标椅子的颜色与训练数据中出现的颜色一致。

表 S6 包含该测试的成功率结果。TCN 和 GRU-ODE 模型在此设置下检测目标极为困难（不到 45% 的运行到达第一个检查点，且没有任何一次运行到达第二个检查点）。液态网络在此场景中表现最佳，其中一个 CfC 在目标时域为三的徒步任务中完成了 20 次运行中的 14 次。该实验的结果有力表明，液态网络能够学习其所给任务的鲁棒表示，并能很好地泛化到其他模型失败或表现不佳的 OOD 场景（关于无人机输入相机感知到的扰动和对抗对象的示例，见图 S7 和 S8）。这一观察与近期研究（21）一致，这些研究表明液态网络是 DCM（43），并且能够为其感知模块学习鲁棒表示，以执行鲁棒的决策。

对象之间的三角循环

在这项任务中，我们证明了我们的方法可以推广到无限多次徒步移动步骤：通过将三把椅子设置为边长为10 m的等边三角形的顶点，如图4所示。我们对四旋翼进行编程，使其在检测到检查点后向右转120°（而不是像上述实验中那样向上飞），以提高下一个目标出现在视野中的可能性。使用CfC架构，我们

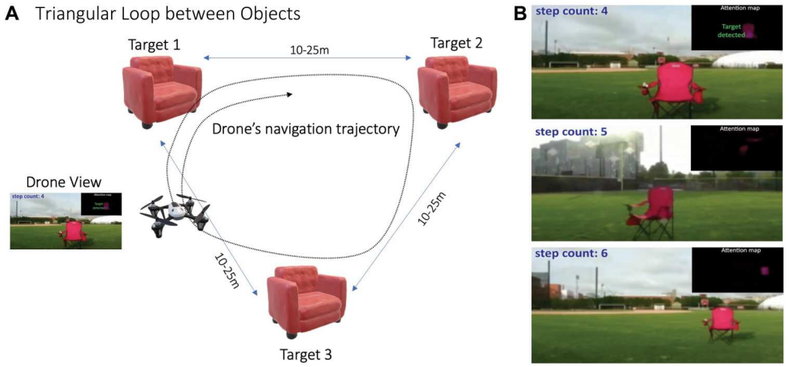


Fig. 4. Triangular loop between objects. (**A**) The triangular infinite loop task setup. (**B**) Time instances of drone views together with their attention map at steps 4, 5, and 6 of the infinite loop.

observed that the network could indefinitely navigate the quadrotor to new targets (we stopped after four laps, with a total of 12 checkpoints reached and detected).

Dynamic target

A useful task in quadrotor autonomous flight is following a moving target. We thus tested the trained policies' ability to pilot the drone in pursuit of a dynamic target. For this task, a figure of eight courses with six checkpoints was set out (at the extremities and either side of each loop, consecutive checkpoints being 5 m apart). This design choice ensured that run lengths were not upper-bounded, tested both turning directions, and included all possible natural lighting angles, all while containing the experiment in a constrained space. The target was moved from one checkpoint to the next, ensuring that enough reaction time in the camera field of view was granted, and the number of checkpoints reached was assigned as the score of a given test run. The experiment was reproduced in two environments, the Urban Lawn setting and a sports field we call Grass Pitch (see fig. S10). Testing in the latter environment is subject to highly challenging conditions of glaring sunlight and strong wind. In such conditions, most network architectures struggled to even latch onto the target (get to the first checkpoint and start following the target). We took this into account in our results by taking averages only on latched test runs but providing the rates at which each network initially detected the target.

Hence, table S7 shows that liquid networks consistently achieved longer trajectories in the Urban Lawn environment, ahead of LSTM and ODE-RNN architectures. In the Grass Pitch setting, all networks apart from LSTM, NCP, and CfC failed to latch onto the target in all attempts. In this challenging environment, liquid architectures marginally outperformed LSTM in terms of trajectory length, although CfC managed to latch onto the target more than twice as often as LSTM and NCP policies.

DISCUSSION

Choice of models matters for OOD generalization

Our experiments show large inequalities between different RNN architectures when performing a diverse range of fly-to-target control tasks in closed loop and all the more so when required to generalize in unseen and adversarial environments. Table 1 suggests that only five of the six networks were capable of reaching the target in the Training Woods environment on half or more of the attempts. TCN (7.5%) failed to achieve this threshold even on data from the training background distribution. These models also performed poorly in the other testing scenarios, with TCN unable to achieve a single success in any of the other three environments and GRU-ODE performing worse in comparison with the Training Woods scenario with the exception of Urban Lawn case, where it succeeded about a third of the time (32.5%). These two architectures exhibited both poor closed-loop performance and generalization and were thus deemed incapable of understanding and performing the control task assigned. Among the models performing reasonably well in both woods scenarios are the ODE-RNNs, with a 62.5% success rate when the target was in the same position as in the training data and up to 82.5% with the target placed in a slightly different position. Although this network seemed to have acquired the capability to achieve the task in closed loop, it generalized poorly to unseen environments, with 17.5 and 25% OOD success rates in the Urban Lawn and Urban Patio experiments.

Other models seemed to perform consistently in the woods, with success rates of 82.5% for LSTM, 100% for NCP, and 85% for CfC on the Training Woods test. In the Alternative Woods scenario, LSTM and both liquid networks (NCPs and CfCs) succeeded in reaching the target at a high success rate of more than 90%, showcasing the acquired ability to learn and execute closed-loop tasks from expert demonstrations.

When asked to generalize performance on the Urban Lawn, the top performer in this environment is CfC, which highly succeeded in attending to the target (90% success). On the task of flying to the target in the presence of natural adversaries and distractions in the extremely heavy distribution shift setup provided by the Urban Patio, LSTM succeeded in only 27.5% of the attempts, whereas both NCP (52.5%) and CfC (67.5%) achieved the best performance.

Furthermore, comprehensive evidence in favor of a crisp advantage for liquid neural networks for closed-loop end-to-end control learning was amassed through our extensive real-world experimental results. Our brain-inspired neural networks, and all the more so our CfC architecture, largely outperformed all other models on all

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

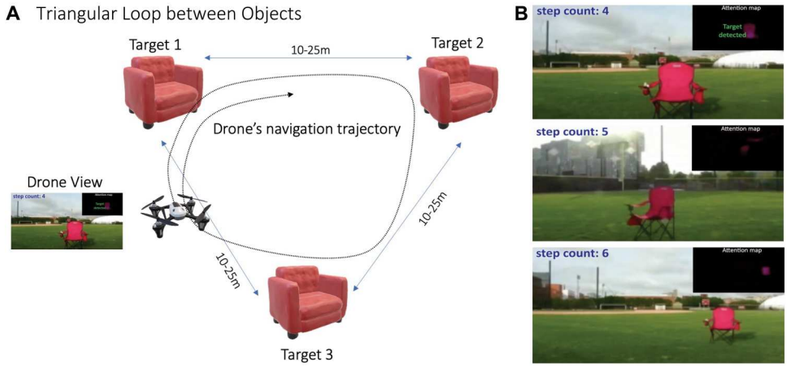


图 4. 物体之间的三角形环路。(A) 三角形无限循环任务设置。(B) 无限循环第 4、5、6 步时无人机视图及其注意力图的时间实例。

观察到该网络可以无限期地引导四旋翼飞向新目标（我们在四圈后停止，总共到达并检测到12个检查点）。

动态目标

一种用在四旋翼自主飞行中，是跟随移动目标。因此，我们测试了训练后的策略操控无人机追踪动态目标的能力。对于该任务，我们布置了一条八字形赛道，设有六个检查点（位于各环路的端点和两侧，相邻检查点相距 5 米）。这一设计选择确保了单次运行的长度没有上限，测试了两种转向方向，并涵盖了所有可能的自然光照角度，同时将实验限制在有限空间内。目标从一个检查点移动到下一个检查点，确保在相机视野中留有足够的反应时间，并将到达的检查点数量作为给定测试运行的得分。该实验在两种环境中重复进行：城市草坪（Urban Lawn）场景以及我们称为草地运动场（Grass Pitch）的运动场（见图 S10）。在后一环境中进行测试时，会遇到刺眼阳光和强风等极具挑战性的条件。在此类条件下，大多数网络架构甚至难以锁定目标（到达第一个检查点并开始跟随目标）。我们在结果中考虑到了这一点：仅对成功锁定目标的测试运行取平均值，同时提供各网络最初检测到目标的比率。

因此，表 S7 显示，液态网络在城市草坪环境中始终能够实现更长的轨迹，领先于 LSTM 和 ODE-RNN 架构。在草地环境中，除 LSTM、NCP 和 CfC 之外，所有网络在所有尝试中均未能锁定目标。在这一具有挑战性的环境中，就轨迹长度而言，液态架构略微优于 LSTM，尽管 CfC 锁定目标的频率是 LSTM 和 NCP 策略的两倍多。

讨论

模型选择对 OOD 泛化至关重要

我们的实验 s[illegible] 之间

在未见过的和对抗性环境中。表 1 表明，在 Training Woods 环境中，六个网络中只有五个能够在半数或更多尝试中到达目标。TCN（7.5%）即使在使用来自训练背景分布的数据时也未能达到该阈值。这些模型在其他测试场景中同样表现不佳：TCN 在其他三个环境中均未能取得一次成功；GRU-ODE 与 Training Woods 场景相比表现更差，但 Urban Lawn 情况除外，在该情况下它约有三分之一的时间（32.5%）取得成功。这两种架构在闭环性能和泛化能力上均表现不佳，因此被认为无法理解并执行所分配的控制任务。在两个 Woods 场景中均表现尚可的模型包括 ODE-RNN；当目标位于与训练数据相同的位置时，其成功率为 62.5%，而当目标置于略有不同的位置时，成功率最高可达 82.5%。尽管该网络似乎已具备在闭环中完成该任务的能力，但其对未见环境的泛化能力较差；在 Urban Lawn 和 Urban Patio 实验中，OOD 成功率分别为 17.5% 和 25%。

其他模型在树林中的表现似乎始终稳定，在 Training Woods 测试中，LSTM 的成功率为 82.5%，NCP 为 100%，CfC 为 85%。在 Alternative Woods 场景中，LSTM 和两个液态网络（NCP 和 CfC）均以超过 90% 的高成功率成功到达目标，展示了从专家演示中学习并执行闭环任务所获得的能力。

在要求泛化到 Urban Lawn 的表现时，该环境中的最佳表现者是 CfC，其在锁定目标方面取得了很高的成功率（90% 成功）。在 Urban Patio 所提供的极端严重分布偏移设置中，面对自然对抗因素和干扰飞向目标的任务上，LSTM 仅在 27.5% 的尝试中成功，而 NCP（52.5%）和 CfC（67.5%）均取得了最佳表现。

此外，我们通过大量真实世界实验结果，积累了全面证据，证明液态神经网络在闭环端到端控制学习方面具有明显优势。我们受大脑启发的神经网络，尤其是我们的 CfC 架构，在所有

of the range tests, the rotation and occlusion robustness tests, the adversarial hiking task, and the dynamic target tracking task.

When all nonliquid networks failed to achieve the task at twice the nominal distance seen in training, our CfC network achieved a 90% success rate and exhibited successful runs even at three times the nominal distance from the target (20%). Also, LSTM′s performance was more than halved when the chair is rotated or occluded, whereas the CfC′s success rate was only reduced by 10% for rotation (with success rates above 80%) and around 30% for occlusion (with success rates at 60%). Similarly, on the adversarial hiking task, CfC achieved a success rate more than twice that of its closest nonliquid competitor, completing 70% of runs, whereas the LSTM model only reached all three targets on 30% of occasions. Last, LSTM could track a moving target for an average of 5.8 steps, whereas our CfC network managed to follow for about 50% longer, averaging 8.8 checkpoints reached. Hence, our brain-inspired networks generalized to both new OOD environments and new tasks better than popular state-of-the-art recurrent neural networks, a crucial characteristic for implementation in real-world applications.

The greater robustness of liquid networks in the face of a data distribution shift was also confirmed through offline stress testing. We note that our brain-inspired networks were overall less prone to drifting from their nominal trajectories in the presence of artificially introduced image perturbations. Our architectures surpassed LSTM on noise, brightness, and contrast perturbations, with the CfC model outperforming all models on each of the perturbations introduced.

Understanding liquid network generalization to OOD settings

We associate the generalization performance of LTCs to two factors. During the end-to-end training, liquid networks impose an inductive bias on the convolutional filters of the head network, which enables a more robust feature representation associated with the task at hand. The saliency maps of the convolutional head networks are great empirical evidence of this fact. Elsewhere, liquid networks are DCMs at their cell level (*21*). The biological priors imposed on liquid networks form a dynamical system that enables mechanisms for regulating external and internal interventions, which are the necessary pillars of causal models (*36*, *45*, *46*). This way, during training, they manage to interact with the training data in a causal fashion to learn representations for both control and perception. In all tasks, we provide extensive evidence of their ability to drop irrelevant features under heavy distribution shifts and execute the task picked up on from training data, in this case, flying to the target regardless of the background profiles and other environmental variables.

Robustness in perception and control

The generalization of end-to-end imitation agents heavily relies on the data seen and the augmentation strategies in place to help encode the task in network weights. We observed that our data augmentation pipeline substantially improves the perception module of all networks and therefore enables them to obtain stable saliency maps with attention centered on the target (figs. S1 to S3). The augmentation, however, does not enable a robust execution of the control task upon change of test environments.

For instance, see the scenarios illustrated in Fig. 3 (B and C), in which the LSTM′s perception module detects the target but still fails to navigate the drone. In these cases, the execution of the navigation task is hindered either by a distraction in the environment or by a discrepancy between the representations learned in the perception module and the recurrent module. This discrepancy might be behind the LSTM agent′s tendency to fly away from the target or to choose an adversarial target instead of the true one detected by its perception module in the first place. With liquid networks, however, we noticed a stronger alignment of the representations between the recurrent and perception modules because the execution of the navigation task was more robust than that of other models.

Causal models in environments with unknown causal structure

The primary conceptual motivation of our work was not causality in the abstract; it was instead task understanding, that is, to evaluate whether a neural model understands the task given from high-dimensional unlabeled offline data. Traditional causal methods are based on known task structure or low-dimensional state spaces (for example, probabilistic graphical models) (*36*), whereas we can incorporate visual input into liquid networks. Therefore, causal modeling frameworks cannot be used in our high-dimensional tasks presented here, where the task structure is unknown. From the causality angle, liquid neural networks are DCMs: This was theoretically shown in Vorbach *et al.* (*21*).

Through extensive end-to-end closed-loop control testings, we showed a strong performance demarcation signal in favor of liquid neural networks against the advanced RNN models. This capacity to capture causality and contexts and learn reasoned representations for control originates from the incorporation of principled mechanisms for information propagation inspired by neural models of biological brains into our learning algorithms.

MATERIALS AND METHODS

Liquid networks: Brain-inspired neural models

A bottom-up approach to building brain-inspired neural models is to examine how neurons interact with each other through synapses in small biological brains (*19*, *20*). We can identify three mechanisms for information propagation that are abstracted away in the current building blocks of neural network-based controllers. First, neural dynamics are typically continuous processes described by differential equations (DEs) (*47*). Second, synaptic release is more than scalar weights: It involves a nonlinear transmission of neurotransmitters, the probability of activation of receptors, and the concentration of available neurotransmitters, among other nonlinearities (*48*). Last, the propagation of information between neurons is induced by feedback and memory apparatuses. These biological priors lead to the design of neural and synapse building blocks at a level of abstraction that is scalable and enriching with neural information processing attributes. The class of liquid neural networks LTC accommodates these three criteria (*35*).

LTCs (Fig. 5) are constructed by the interaction of leaky integrator neural models (*49*) with the steady-state dynamics of a conductance-based nonlinear synapse model (*47*). The model is a differentiable dynamical system with an input-dependent varying (for example, liquid) time characteristic. Their outputs are computed by numerical DE solvers when described by ordinary DEs and by continuous functions when described in closed-form

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

范围测试、旋转和遮挡鲁棒性测试、对抗性徒步任务以及动态目标跟踪任务中，都大幅优于所有其他模型。

当所有非液态网络在训练中所见标称距离的两倍处都无法完成该任务时，我们的 CfC 网络达到了 90% 的成功率，并且即使在距目标标称距离三倍处也表现出成功运行（成功率为 20%）。此外，当椅子被旋转或被遮挡时，LSTM 的性能下降了一半以上，而 CfC 的成功率在旋转时仅下降 10%（成功率高于 80%），在遮挡时仅下降约 30%（成功率为 60%）。同样，在对抗性徒步任务中，CfC 的成功率是其最接近的非液态竞争对手的两倍以上，完成了 70% 的运行；而 LSTM 模型仅在 30% 的情况下到达全部三个目标。最后，LSTM 平均能够跟踪移动目标 5.8 步，而我们的 CfC 网络跟踪时间约长 50%，平均到达 8.8 个检查点。因此，我们的类脑网络在泛化到新的 OOD 环境和新的任务方面，都优于流行的最先进循环神经网络，这是在实际应用中部署的一项关键特性。

液态网络在面对数据分布偏移时更强的鲁棒性，也通过离线压力测试得到了证实。我们注意到，在人为引入图像扰动的情况下，我们的类脑网络总体上更不容易偏离其标称轨迹。我们的架构在噪声、亮度和对比度扰动方面均优于 LSTM，其中 CfC 模型在每一种引入的扰动下都优于所有模型。

理解液态网络在 OOD 设置下的泛化

我们将 LTCs 的泛化性能与两个因素联系起来。在端到端训练期间，液态网络会对头部网络的卷积滤波器施加归纳偏置，从而形成与当前任务相关的更稳健的特征表示。卷积头部网络的显著性图是这一事实的有力经验证据。另一方面，液态网络在其单元层面上是 DCM（21）。施加于液态网络的生物先验构成一个动力系统，该系统促成了调节外部和内部干预的机制，而这些机制是因果模型的必要支柱（36, 45, 46）。这样一来，在训练期间，它们能够以因果方式与训练数据交互，从而学习用于控制和感知的表示。在所有任务中，我们都提供了大量证据，证明它们能够在严重分布偏移下丢弃无关特征，并执行从训练数据中学到的任务；在本例中，就是无论背景轮廓和其他环境变量如何，都能飞向目标。

感知与控制中的鲁棒性

端到端模仿智能体的泛化能力在很大程度上依赖于所见的数据以及用于帮助将任务编码到网络权重中的增强策略。我们观察到，我们的数据增强流程显著改善了所有网络的感知模块，从而使它们能够获得稳定的显著性图，并将注意力集中在目标上（图 S1 至 S3）。然而，当测试环境发生变化时，增强并不能使控制任务得到稳健执行。

例如，参见图 3（B 和 C）所示的情景，其中 LSTM 的感知模块检测到了目标，但仍然失败无法操控无人机完成导航。在这些情况下，导航任务的执行会受到阻碍：要么是由于环境中的干扰，要么是由于感知模块与循环模块所学到的表征之间存在差异。这种差异可能正是 LSTM 智能体倾向于飞离目标，或者选择对抗性目标而不是其感知模块最初检测到的真实目标的原因。然而，使用液态网络时，我们注意到循环模块与感知模块之间的表征具有更强的一致性，因为导航任务的执行比其他模型更稳健。

未知因果结构环境中的因果模型

我们工作的首要概念动机并不是抽象的因果性；而是任务理解，即评估神经模型是否理解从高维、无标签的离线数据中给出的任务。传统因果方法基于已知任务结构或低维状态空间（例如概率图模型）（36），而我们能够将视觉输入纳入液态神经网络。因此，因果建模框架无法用于本文所呈现的高维任务，因为这些任务的结构是未知的。从因果性的角度看，液态神经网络是 DCMs：这一点已在 Vorbach 等（21）中得到理论证明。

通过广泛的端到端闭环控制测试，我们发现了明显偏向液体神经网络的性能分界信号，其表现优于先进的 RNN 模型。这种捕捉因果关系与上下文、并学习用于控制的推理性表征的能力，源于将受生物大脑神经模型启发的、有原则的信息传播机制融入我们的学习算法。

材料与方法

液体网络：受大脑启发的神经模型

构建受大脑启发的神经模型的一种自下而上的方法，是考察小规模生物大脑中神经元如何通过突触彼此交互（19, 20）。我们能够识别出三种信息传播机制，而当前基于神经网络的控制器构建模块将这些机制抽象掉了。第一，神经动力学通常是由微分方程（DE）描述的连续过程（47）。第二，突触释放不仅仅是标量权重：它涉及神经递质的非线性传递、受体激活概率以及可用神经递质的浓度，还有其他非线性因素（48）。最后，神经元之间的信息传播由反馈和记忆装置所诱导。这些生物学先验促成了神经与突触构建模块的设计，其抽象层次具有可扩展性，并丰富了神经信息处理属性。液体神经网络 LTC 这一类别满足了这三项标准（35）。

LTC（图5）由漏积分器神经模型（49）与基于电导的非线性突触模型（47）的稳态动力学相互作用而构建。该模型是一个可微动力系统，具有依赖输入而变化的（例如“液体”）时间特性。当其由常微分方程描述时，其输出由数值微分方程求解器计算；当其以闭式近似描述时，则通过连续函数计算（图5B）（50）。

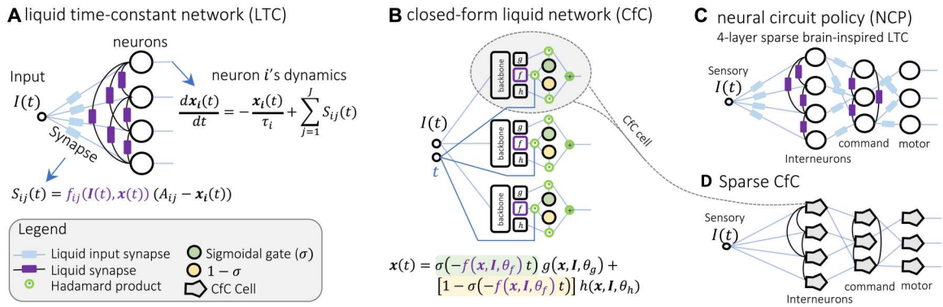


Fig. 5. Liquid neural networks. (**A**) Schematic demonstration of a fully connected LTC layer (*35*). The dynamics of a single neuron *i* is given, where $\mathbf{x}_i(t)$ represents the state of the neuron *i* and τ represents the neuronal time constant. Synapses are given by a nonlinear function, where *f* (.) represents a sigmoidal nonlinearity and A_i *j* is a reversal potential parameter for each synapse from neurons *j* to *i*. (**B**) Schematic representation of a closed-form liquid network (CfC) together with its state equation. This continuous-time representation consists of two sigmoidal gating mechanisms, σ and 1 − σ, with their activity being regulated by the nonlinear neural layer *f*. *f* acts as the LTC for CfCs. These gates control two other nonlinear layers, *g* and *h*, to create the state of the system. (**C**) A schematic representation of an NCP (*18*). The network architecture is inspired by the neural circuits of the nematode *C. elegans*. It is constructed by four sparsely connected LTC layers called sensory neurons, interneurons, command neurons, and interneurons. (**D**) CfC. A sparse neural circuit with the same architectural motifs as NCPs, this time with CfC neural dynamics.

approximations (Fig. 5B) (*50*). These brain-inspired models are instances of continuous-time (CT) neural networks (*35*, *41*) that can be trained via gradient descent in modern automatic differentiation frameworks. Liquid networks exhibit stable and bounded behavior, yield superior expressivity within the family of CT neural models (*35*, *41*), and give rise to improved performance on a wide range of time series prediction tasks compared with advanced, recurrent neural network models (*50*). In particular, a sparse network configuration composed of fewer than two dozen LTC neurons supplied with convolutional heads showed great promise in learning, end to end, to map high-dimensional visual input stream of pixels to robust control decisions (*18*). These liquid network instances are called NCPs, because their four-layered network structure is inspired by the neural circuits of the nervous system of the nematode *C. elegans* (*51*), as illustrated in Fig. 5 (C and D). In prior work, these sparse liquid networks learned how to navigate autonomous simulated aerial (*21*) and real ground (*18*) vehicles to their goals much more robustly than their advanced deep learning counterparts in a large series of behavioral-cloning experiments within their training distribution. The state-space representation of an LTC neural network is determined by the following set of ODEs (*35*):

$$\frac{d\mathbf{x}(t)}{dt} = -\left[\frac{1}{\tau} + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)\right] \odot \mathbf{x}(t) + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta) \odot A \tag{1}$$

Here, $\mathbf{x}^{(D\times 1)}(t)$ is the hidden state with size D, $\mathbf{I}^{(m\times 1)}(t)$ is an input signal, $\tau^{(D\times 1)}$ is the fixed internal time-constant vector, $A^{(D\times 1)}$ is a bias parameter, and ⊙ is the Hadamard product. Intuitively, LTC networks are able to change their equations on the basis of the input they observe. These networks, either in their ODE form or in their closed-form representation (*50*), demonstrate causality and generalizability in modeling spatiotemporal dynamics compared with their counterparts (*21*). Their closed-form representations are called closed-form continuous-time (CfC) models and are given by (*50*):

$$\mathbf{x}(t) = \underbrace{\sigma(-f(\mathbf{x},\mathbf{I};\theta_f)\mathbf{t})}_{\text{Time-continuous gating}} \odot g(\mathbf{x},\mathbf{I};\theta_g) + \underbrace{[1-\sigma(-[f(\mathbf{x},\mathbf{I};\theta_f)]\mathbf{t})]}_{\text{Time-continuous gating}} \odot h(\mathbf{x},\mathbf{I};\theta_h) \tag{2}$$

Here, *f*, *g*, and *h* are three neural network heads with a shared backbone, parameterized by θ_f, θ_g, and θ_h, respectively. $\mathbf{I}(t)$ is an external input, and *t* is the time sampled by input time stamps, as illustrated in Fig. 5.

Throughout, liquid (neural) networks refer to the general category of models that are presented either by LTCs or CfCs. CfCs are closed-form liquid networks, and LTCs are ODE-based liquid networks.

Liquid networks capture causality

The key to liquid networks' robust performance under distribution shifts is their ability to dynamically capture the true cause and effect of their given task (*21*). This can be shown analytically because LTCs are DCMs (*21*, *43*), a framework through which models can account for internal and external interventions with independent mechanisms. Vorbach *et al.* (*21*) theoretically showed that the learning system described by an LTC network of Eq. 1 can control internal and external interventions by the network parameters θ and thus reduces to a DCM (*43*) as long as *f* is monotonically increasing, bounded, and Lipschitz continuous.

DCMs are probabilistic graphical models (*43*). DCMs differ from other causal models in that they do not cast around statistical dependencies from data directly; rather, they have dynamic mechanisms, such as the structure presented in Eq. 2, that enable extracting cause-and-effect from data (*52*).

Causal properties of DCMs focus the attention of LTCs on the task rather than the context of the task, and, for this reason, in this article, we hypothesize and show that tasks learned in one environment can be transferred to different environments for LTC networks where other models fail. More formally, consider a sequence of task steps **T**, a sequence of robot/world configurations

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

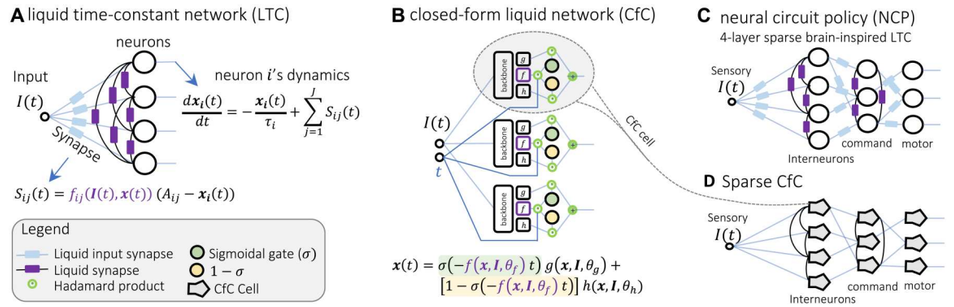


图5. 液态神经网络。(A) 全连接LTC层（35）的示意图。给出了单个神经元i的动力学，其中 x_i(t) 表示神经元i的状态， τ 表示神经元时间常数。突触由非线性函数给出，其中 f (.) 表示S型非线性，A_i j 是从神经元j到i的每个突触的反转电位参数。(B) 闭式液态网络（CfC）及其状态方程的示意图。这种连续时间表示由两个S型门控机制 σ 和 1 − σ 组成，其活动受非线性神经层 f 调控。f 充当 CfC 的 LTC。这些门控控制另外两个非线性层 g 和 h，以形成系统的状态。(C) NCP（18）的示意图。该网络架构受线虫秀丽隐杆线虫（C. elegans）神经回路的启发。它由四个稀疏连接的LTC层构成，分别称为感觉神经元、中间神经元、指令神经元和中间神经元。(D) CfC。一个具有与NCP相同架构基序的稀疏神经回路，这一次采用CfC神经动力学。

近似（图5B）（50）。这些受大脑启发的模型是连续时间（CT）神经网络（35，41）的实例，可在现代自动微分框架中通过梯度下降进行训练。液体网络表现出稳定且有界的行为，在CT神经模型家族中具有更优越的表达能力（35，41），并且在广泛的时间序列预测任务上，相比先进的循环神经网络模型，能够带来性能提升（50）。特别地，一种由不到两打LTC神经元并配以卷积头组成的稀疏网络配置，在端到端学习将高维像素视觉输入流映射为稳健控制决策方面展现出巨大前景（18）。这些液体网络实例被称为NCP，因为其四层网络结构受到线虫C. elegans神经系统的神经环路启发（51），如图5（C和D）所示。在先前的工作中，这些稀疏液体网络在训练分布内进行的一系列大规模行为克隆实验中，学会了如何将自主模拟空中（21）和真实地面（18）车辆导航至目标，且比先进的深度学习对应模型稳健得多。LTC神经网络的状态空间表示由以下一组常微分方程（ODE）确定（35）：

$$\frac{d\mathbf{x}(t)}{dt} = -\left[\frac{1}{\tau} + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)\right] \odot \mathbf{x}(t) + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta) \odot A \tag{1}$$

这里，$x^{(D\times 1)}$(t) 是大小为 D 的隐藏状态，$I^{(m\times 1)}$(t) 是输入信号，$\tau^{(D\times 1)}$ 是固定的内部时间常数向量，$A^{(D\times 1)}$ 是偏置参数，⊙ 是 Hadamard 积。直观地说，LTC 网络能够根据其观测到的输入来改变其方程。这些网络无论是采用 ODE 形式，还是采用其闭式表示 (50)，与对应模型 (21) 相比，在建模时空动力学时都表现出因果性和泛化能力。它们的闭式表示被称为闭式连续时间 (CfC) 模型，并且

are given by (*50*):

$$\mathbf{x}(t) = \underbrace{\sigma(-f(\mathbf{x},\mathbf{I};\theta_f)\mathbf{t})}_{\text{Time-continuous gating}} \odot g(\mathbf{x},\mathbf{I};\theta_g) + \underbrace{[1-\sigma(-[f(\mathbf{x},\mathbf{I};\theta_f)]\mathbf{t})]}_{\text{Time-continuous gating}} \odot h(\mathbf{x},\mathbf{I};\theta_h) \tag{2}$$

这里，f、g 和 h 是三个具有共享骨干网络的神经网络头，分别由θ_f、 θ_g 和 θ_h 参数化。I(t) 是外部输入，t 是由输入时间戳采样得到的时间，如图 5 所示。

在全文中，液态（神经）网络指的是由LTC或CfC所表示的一类通用模型。CfC是闭式液态网络，LTC是基于ODE的液态网络。

液态网络捕捉因果关系

液态网络在分布偏移下具有稳健性能的关键，在于其能够动态捕捉给定任务的真实因果（21）。这一点可以在分析上得到证明，因为LTC是DCM（21, 43）；DCM是一个框架，通过该框架，模型能够以相互独立的机制来解释内部和外部干预。Vorbach等人（21）从理论上表明，只要f单调递增、有界且Lipschitz连续，由式1的LTC网络所描述的学习系统就可以通过网络参数 θ 控制内部和外部干预，并因此归约为DCM（43）。

DCM是概率图模型（43）。DCM与其他因果模型的不同之处在于，它们并不直接围绕数据中的统计依赖关系展开；相反，它们具有动态机制，例如式2所示的结构，从而能够从数据中提取因果关系（52）。

DCM 的因果特性使 LTC 将注意力集中在任务本身，而非任务的上下文上；因此，在本文中，我们提出假设并表明，对于 LTC 网络，在一个环境中学习到的任务可以迁移到不同环境，而其他模型在这种情况下会失败。更形式化地说，考虑任务步骤序列 T、机器人/世界配置序列

C, and a sequence of images **V**. Visual data are generated following a schematic graphical model:

$$\mathbf{T} \rightarrow \mathbf{C} \rightarrow \mathbf{V}$$

We can then explain how causal understanding implies knowing. Robot states cause images and not vice versa (so when training on images and control inputs, the network should output positive velocity when required for the task, not because it has learned that a sequence of images moving forward implies that the drone is moving forward). In addition, the drone's motion is governed by the task (for example, centering the chair in the frame) and not by other visual correlations in the data. With the data augmentation techniques implemented, we show that almost all networks can handle the former condition. The latter condition, however, is difficult to achieve for many networks, as extensively exhibited throughout this report. Our objective in designing fly-to-target tasks is to create a testbed where neither the concept of objects nor the task itself is given to the agent. The agents must learn/understand the task, the scene, and the object from offline data without any guidance on what the scene, object, and task are. In this setting, supervised learning of object detection, where we have to manually label bounding boxes, is out of scope because we aim explicitly not to use labeled data (bounding boxes). We show how to learn the task and scene end to end from data and generalize to unseen and heavily shifted environments.

To this end, we set out to empirically assess this property in a large series of in- and out-of-distribution end-to-end navigation tasks in challenging environments. We built advanced neural control agents for autonomous drone navigation tasks (fly-to-target). We explored their generalization capabilities in new environments with a drastic change of scenery, weather conditions, and other natural adversaries.

Training procedure

The following section details the preparation of the data and hyperparameters used for training the onboard models.

Data preparation

The training runs were originally collected as long sequences in which the drone moved between all five targets. Because sequences of searching for the next target and traveling between them could lead to ambiguity in the desired drone task, for each training run of five targets, we spliced the approach to each of the five targets into five separate training sequences and added these new sliced sequences to the training data.

Data augmentation

To increase the networks' robustness and generalization capabilities, we performed various image augmentations during training. The first set of augmentations were a random brightness shift with brightness offsets selected uniformly at random ranging from 0 to 0.4× of the max pixel brightness, a random contrast shift with a contrast factor between 0.6 and 1.4, and a random saturation multiplication between 0.6 and 1.4. Parameters for the aforementioned image augmentations were fixed within a sequence but randomized between sequences so that two images within the same training sequence had identical brightness, contrast, and

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

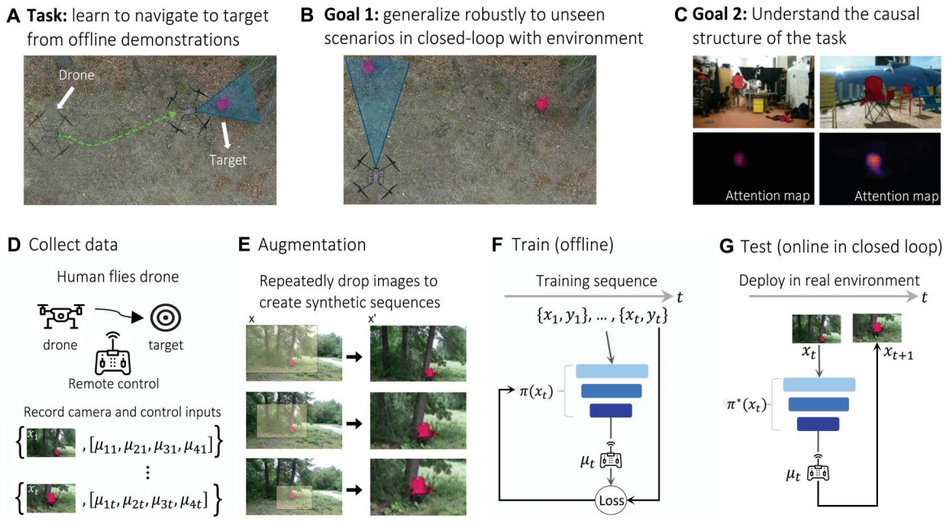


Fig. 6. End-to-end learning setup. (**A**) Policies were trained to solve the following task: Using only images taken from an onboard camera, navigate the drone from its starting location to a target 10 m away, keeping the object in the frame throughout. (**B**) One goal of the work is to explore the generalization capability of various neural architectures by adapting the task to previously unseen environments. (**C**) Another goal is to understand the causal mechanisms underlying different networks' behaviors while completing the task, by visualizing networks' input saliency maps. (**D**) The training process started by hand-collecting human expert trajectories and recording camera observations and human expert actions. (**E**) To further convey the task, we then generated synthetic sequence by taking images collected during human flights and repeatedly cropping them, creating the appearance of a video sequence in which the drone flies up to the target and centers it in the frame. (**F**) The networks were trained offline on the collected expert sequences using a supervised behavior cloning MSE loss. (**G**) We then deployed trained networks on the drone in online testing and observed task performance under heavy distributional shifts, varying lighting, starting location, wind, background setting, and more.

C，以及图像序列 V。视觉数据按照如下示意性图模型生成：

$$\mathbf{T} \rightarrow \mathbf{C} \rightarrow \mathbf{V}$$

然后，我们可以解释因果理解如何意味着知晓。机器人状态导致图像，而不是反过来（因此，在基于图像和控制输入进行训练时，网络应在任务需要时输出正向速度，而不是因为它已经学会：一系列向前移动的图像意味着无人机正在向前移动）。此外，无人机的运动由任务支配（例如，将椅子置于画面中心），而不是由数据中的其他视觉相关性支配。通过所实施的数据增强技术，我们表明几乎所有网络都能处理前一种条件。然而，后一种条件对许多网络来说难以实现，正如本报告通篇广泛展示的那样。我们设计飞向目标（fly-to-target）任务的目标是创建一个测试平台，其中既不会向智能体提供物体概念，也不会提供任务本身。智能体必须从离线数据中学习/理解任务、场景和物体，而无需任何关于场景、物体和任务是什么的指导。在这种设定下，物体检测的监督学习（其中我们必须手动标注边界框）不在范围内，因为我们明确旨在不使用标注数据（边界框）。我们展示了如何从数据中端到端地学习任务和场景，并泛化到未见过的且分布严重偏移的环境。

为此，我们着手在一系列大规模的分布内和分布外端到端导航中，对这一性质进行实证评估

具有挑战性环境中的任务。我们为自主无人机导航任务（飞向目标）构建了先进的神经控制智能体。我们探索了它们在场景、天气条件及其他自然不利因素发生剧烈变化的新环境中的泛化能力。

训练流程

以下部分详细介绍了用于训练机载模型的数据准备和超参数。

数据准备

训练运行最初是以长序列的形式采集的，在这些序列中无人机在全部五个目标之间移动。由于搜索下一个目标并在目标之间飞行的序列可能会使期望的无人机任务产生歧义，因此对于每次包含五个目标的训练运行，我们将接近五个目标中每一个目标的过程切分成五个独立的训练序列，并将这些新切分出的序列加入训练数据。

数据增强

为了提高网络的鲁棒性和泛化能力，我们在训练过程中进行了多种图像增强。第一组增强操作包括随机亮度偏移，其亮度偏移量在 0 到最大像素亮度的 0.4× 之间均匀随机选取；随机对比度偏移，其对比度因子在 0.6 到 1.4 之间；以及随机饱和度乘法，其乘数在 0.6 到 1.4 之间。上述图像增强操作的参数在一个序列内固定，但在不同序列之间随机化，因此同一训练序列中的两张图像具有相同的亮度、对比度和

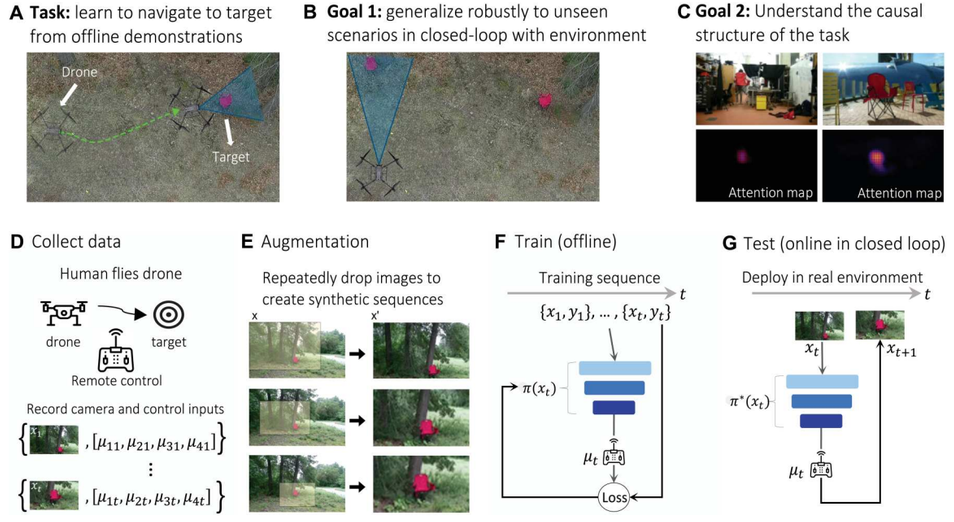


图6. 端到端学习设置。(A) 训练策略以解决以下任务：仅使用机载摄像头拍摄的图像，将无人机从其起始位置导航至10米远的目标，并在整个过程中将该目标保持在画面中。(B) 这项工作的一项目标是通过将该任务适配到先前未见过的环境，探索各种神经网络架构的泛化能力。(C) 另一项目的是通过可视化网络的输入显著性图，理解不同网络在完成任务时行为背后的因果机制。(D) 训练过程始于手工采集人类专家轨迹，并记录摄像头观测和人类专家动作。(E) 为了进一步呈现该任务，我们随后利用人类飞行期间采集的图像并通过反复裁剪这些图像生成合成序列，从而营造出无人机飞向目标并将其置于画面中央的视频序列效果。(F) 网络使用监督式行为克隆MSE损失，在采集的专家序列上进行离线训练。(G) 随后，我们在无人机上部署训练好的网络进行在线测试，并在严重的分布偏移、变化的光照、起始位置、风、背景设置等条件下观察任务性能。

saturation offsets, but two images in different sequences had different offsets. Last, each image had Gaussian random noise with a mean of 0 and an SD of 0.05 added.

In addition, to better convey the task, we performed closed-loop augmentation by generating synthetic image and control sequences and adding them to the training set. To generate these sequences, we took a single image with the target present and repeatedly cropped the image. Over the duration of the synthetic sequence, we moved the center of the crop from the edge of the image to the center of the target, causing the target to appear to move from the edge of the frame to the center. In addition, we shrank the size of the cropped window and upscaled all of the cropped images to the same size, causing the target to appear to grow bigger. Together, these image procedures simulated moving the drone to center the target while moving closer to the target to make it occupy more of the drone′s view. We then inserted still frames of the central target with a commanded velocity of zero for 20 to 35% of the total sequence length. This pause sought to teach the drone to pause after centering the target. Algorithm S1 describes the synthetic sequence generation steps, whereas Fig. 6E visualizes the successive crops calculated by the algorithm and their resulting output frames.

To label synthetic sequences with velocity controls, we generated yaw commands equal to the horizontal distance in pixels between the target and the center of the frame times 0.01 and likewise generated throttle (up-down) commands equal to the vertical target offset times 0.01. We generated pitch (forward-backward) commands in the synthetic data proportional to the size of the cropped window. This technique increased the size of the training set by more than 2.5×, using a relatively small sample of images in which the target was present.

The initial position of the target within the frame was chosen randomly such that the x (horizontal) offset of the chair was between 10 and 70 pixels, and the y offset was between 5 and 40 pixels. (The size of the input images was 144 pixels by 256 pixels.] Synthetic sequences were later balanced such that the mean control signal across all generated data points was zero for the throttle and yaw channels. The sequence length was also randomized to be between 120 and 250 frames to encourage robustness to external forces or varying update frequencies. Note that although we used these augmented sequences for training, we did not include the closed-loop synthetic sequences in the validation set.

Hyperparameter tuning/network architectures

For each network architecture, we used tree-structured Parzen estimators (TPEs) (*53*, *54*) for tuning hyperparameters such as learning rate (LR), LR decay rate, and model-specific hyperparameters such as backbone unit count or connection seed that would maximize performance. The set of parameters leading to the lowest train + validation loss over the 40 episodes was selected for the final tested model.

TPE, a sequential model–based optimization algorithm, was responsible for sampling new hyperparameters to try across different episodes. After three trials, episodes with a higher median training + validation loss on the 10th epoch were pruned early and did not run to completion. Hyperparameter optimization was implemented using the Optuna library. Not all parameters were found via hyperparameter tuning; some parameters were hand-selected to shrink the dimensionality of the search space. The best hyperparameters found for each model architecture and used during testing are listed below in table S1.

For each chosen hyperparameter configuration, the number of trainable parameters in the corresponding model for every neural architecture is listed in table S3. Each network tested was prefixed by a simple CNN backbone for processing incoming images. The 128-dimensional CNN features were then fed to the recurrent unit that predicted control outputs. The shared CNN architecture is pictured in table S4.

Fine-tuning

To simultaneously learn the geospatial constructs present in the long, uncut sequences and the task-focused controls present in the cut and synthetic sequences, we fine-tuned a model trained on the long sequences with sequences featuring only one target. The starting checkpoint was trained on the original uncut sequences, all sliced sequences, and synthetic data corresponding to all targets. We then regenerated synthetic data with new randomized offsets for one target and fine-tuned on sliced sequences containing the one target and new synthetic augmented sequences that featured that target. Training parameters are listed in table S2.

Platform setup

We evaluated the proposed RNN architectures on a commercially available quadcopter with custom software running onboard. The system featured a forward-facing gimbal-mounted camera and a GPU to run onboard inference. We used Robot Operating System (ROS) (*55*) to handle communication between the RNN inference code and the lower-level drone flight controller.

Drone hardware

A DJI M300 RTK quadcopter was used for data collection and evaluation of policy performance. The M300 is an enterprise-grade drone that can be flown by manual control in its default configuration. It also interfaces with the DJI Manifold 2 companion computer, enabling programmatic control of the drone. The companion computer included an NVIDIA Jetson TX2, which has a CPU and GPU. The DJI Onboard SDK and its associated ROS wrapper provided an interface for giving the drone′s low-level flight controller desired linear and angular velocity set points. The flight controller is a black box not modifiable by the end user, but it controls the four-rotor speeds to track the velocities specified by the TX2 computer.

A Zenmuse Z30 camera and gimbal were mounted on the underside of the drone, pointed forward. The gimbal compensated for the drone′s current orientation such that the roll or pitch of the drone did not result in roll or pitch of the camera image. The gimbal followed the drone′s current yaw such that the camera was always pointed forward. Images from the camera were available to the companion computer via the SDK.

System design

We implemented our software control system in ROS. A ROS node received images from the onboard camera, ran inference with the RNN controller to compute the new desired control on the basis of the current image, and sent the generated control outputs to the low-level flight controller using the onboard ROS SDK. The node has two threads: One is responsible for processing incoming images, and the other is responsible for running inference on the images and sending the resulting control signals to the flight controller. The TX2′s GPU ran the RNN inference with GPU-accelerated Tensorflow 2.3.x. A depiction of the system design can be found in fig. S2.

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

饱和度偏移量，但不同序列中的两张图像具有不同的偏移量。最后，每张图像都加入了均值为 0、标准差为 0.05 的高斯随机噪声。

此外，为了更好地传达任务，我们通过生成合成图像和控制序列并将其添加到训练集中，执行了闭环增强。为了生成这些序列，我们取一张包含目标的单张图像并反复裁剪该图像。在合成序列的持续时间内，我们将裁剪的中心从图像边缘移动到目标中心，使目标看起来从画面边缘移动到中心。此外，我们缩小了裁剪窗口的尺寸并将所有裁剪图像放大到相同大小，使目标看起来变大。这些图像处理操作共同模拟了将无人机移动到目标居中位置的同时靠近目标，使其占据更多无人机视野。然后，我们在总序列长度的20%至35%中插入中心目标的静止帧，并命令速度为零。这一暂停旨在教会无人机在将目标居中后暂停。算法S1描述了合成序列生成步骤，而图6E可视化了该算法计算出的连续裁剪及其产生的输出帧。

为了给合成序列标注速度控制，我们生成的偏航指令等于目标与画面中心之间的水平像素距离乘以 0.01；同样，我们生成的油门（上下）指令等于目标的垂直偏移量乘以 0.01。我们在合成数据中生成的俯仰（前后）指令与裁剪窗口的大小成正比。该技术使训练集的规模增加了超过 2.5× 倍，而只使用了相对较少的包含目标的图像样本。

目标在画面中的初始位置是随机选择的，使椅子的 x（水平）偏移量在 10 到 70 像素之间，y 偏移量在 5 到 40 像素之间。（输入图像的大小为 144 像素 × 256 像素。]之后，合成序列经过平衡，使油门和偏航通道在所有生成数据点上的平均控制信号为零。序列长度也被随机设定在 120 到 250 帧之间，以增强对外部作用力或更新频率变化的鲁棒性。请注意，尽管我们将这些增强序列用于训练，但我们并未将闭环合成序列纳入验证集。

超参数调优/网络架构

针对每种网络架构，我们使用树结构 Parzen 估计器（TPEs）(53, 54) 来调优超参数，例如学习率（LR）、LR 衰减率，以及模型特定的超参数（如骨干单元数量或连接种子），以最大化性能。我们选择在 40 个回合中使训练 +验证损失最低的参数集，用于最终测试的模型。

TPE 是一种基于序列模型的优化算法，负责为不同回合采样新的超参数进行尝试。经过三次试验后，在第 10 个 epoch 上具有较高训练+ 验证损失中位数的回合被提前剪枝，并且未运行至完成。超参数优化使用 Optuna 库实现。并非所有参数都是通过超参数调优找到的；有些参数是手工选择的，以缩小搜索空间的维度。为每种模型架构找到并在测试期间使用的最佳超参数列于下方表 S1 中。

对于每种选定的超参数配置，相应模型在每种神经网络架构下的可训练参数数量列于表 S3 中。所测试的每个网络均前置了一个简单的 CNN 主干网络，用于处理输入图像。随后，128 维 CNN 特征被送入循环单元，由该单元预测控制输出。共享的 CNN 架构展示在表 S4 中。

微调

为了同时学习长且未切割序列中存在的地理空间结构，以及切割序列和合成序列中存在的以任务为重点的对照，我们使用仅包含一个目标的序列，对在长序列上训练的模型进行了微调。起始检查点是在原始未切割序列、所有切片序列以及对应于所有目标的合成数据上训练得到的。随后，我们为一个目标重新生成了具有新随机偏移的合成数据，并在包含该目标的切片序列以及带有该目标的新合成增强序列上进行微调。训练参数列于表 S2 中。

平台搭建

我们在搭载定制机载软件的市售四旋翼无人机上评估了所提出的 RNN 架构。系统配备了一个朝前安装的云台相机和一块用于运行机载推理的 GPU。我们使用机器人操作系统（ROS）（55）来处理 RNN 推理代码与底层无人机飞控之间的通信。

无人机硬件

数据采集和策略性能评估使用了 DJI M300 RTK 四旋翼无人机。M300 是一款企业级无人机，在默认配置下可通过手动控制飞行。它还可与 DJI Manifold 2 配套计算机对接，从而实现对无人机的程序化控制。该配套计算机包含一块 NVIDIA Jetson TX2，其具有 CPU 和 GPU。DJI Onboard SDK 及其相关 ROS 封装提供了接口，用于向无人机的底层飞行控制器给出期望的线速度和角速度设定点。飞行控制器是一个最终用户无法修改的黑盒，但它控制四个旋翼的转速，以跟踪 TX2 计算机指定的速度。

Zenmuse Z30 相机和云台安装于无人机底部，朝向正前方。云台补偿了无人机当前姿态，使无人机的横滚或俯仰不会导致相机图像出现横滚或俯仰。云台跟随无人机当前偏航角，使相机始终朝向正前方。相机图像可通过 SDK 提供给机载伴随计算机。

系统设计

我们在 ROS 中实现了软件控制系统。一个 ROS 节点从机载相机接收图像，使用 RNN 控制器运行推理，以基于当前图像计算新的期望控制，并通过机载 ROS SDK 将生成的控制输出发送到低层飞行控制器。该节点有两个线程：一个负责处理传入图像，另一个负责对图像运行推理并将得到的控制信号发送到飞行控制器。TX2 的 GPU 使用 GPU 加速的 Tensorflow 2.3.x 运行 RNN 推理。系统设计示意图见图 S2。

Analysis methodology

VisualBackprop. A primary goal of this article was to analyze the task understanding of the policy networks. We visually analyzed the task understanding of a policy network by examining how the policy′s CNN encodes and extracts features from an input. Each of the policies shares the same CNN head architecture, so differences in CNN weights between policies are representative of what the downstream architecture learns to extract from the environment.

Using the VisualBackProp algorithm, we used observations of the environment to compute attention mappings from the CNN head of the model architectures. The procedure, shown in Algorithm S4, is as follows: As input was passed through the CNN, each of the feature maps was averaged across the channels and saved. Starting from the final layer, the averaged feature map was deconvolved to the size of the previous layer′s average feature map. The deconvolved feature map was multiplied by the previous layer′s average feature map. This product was then passed to the previous layer. The deconvolution and multiplication were repeated iteratively, updating at each step until it reached the first layer. The result was a feature map displaying locations in the source image to which the model attends the most attention. These maps are the saliency maps (*56*).

Stress tests. For the stress tests in Fig. 2, we averaged the trajectory deviations for 10 runs recorded by the networks running in closed loop. Five runs were taken from the training environment, and five were taken from the urban test task. The perturbed runs were collected across a variety of different network types, including both liquid networks and conventional networks.

We only fed 40% of the frames in each run before measuring the final distance, because the longer the sequence, the more the trajectories diverge. Because the input images fed to the model were fixed, we could not model the change in perspective caused by the diverging viewpoints, and any estimates of future position became more inaccurate. To limit the effect of the offline frame generation while still maintaining enough frames to yield statistically significant results, we chose to only keep the first 40% of each sequence.

Attention-based hiking task setup. We designed a controller that relies on the attention maps perceived by the convolutional layers of each pretrained network architecture to trigger a hop between two targets. We observed that for networks with some understanding of the task when the target of interest is close enough, the saliency map displays a bright patch corresponding to the position of the object in the input image. We thus used an area occupancy criterion to trigger a switch of the target. While the network was piloting the quadrotor, we analyzed each saliency frame, masking pixels under a brightness threshold and detecting the most salient and largest connected patch. When the best candidate patch surpassed a fixed threshold in pixel area (corresponding to the drone being 2 m away from the camping chair), we considered the target reached and triggered an override of the network commands with a fixed-time throttle command to fly the quadrotor upward 1.6 m. This ensured that the current target left the field of view and that the network had access to images of the next segment of the scene. Control was given back to the network until a new target was detected and so on. This control protocol (depicted in fig. S3) can, by design, attend to an indefinite number of checkpoints, provided that the network sees and is capable of navigating to the next target.

Statistical analysis

The quantitative results of the paper presented in Tables 1 and 2 and table S4 to S7 all supply the number of active tests performed, *N*, and report the success rate of models in each scenario in percentages. On all compartments of Fig. 2, each point represents the mean across 10 runs, whereas the vertical bar on each point represents the SD corresponding to the data entry.

Supplementary Materials

This PDF file includes:
Materials and Methods
Figs. S1 to S10
Tables S1 to S7
References (*57*–*61*)

Other Supplementary Material for this manuscript includes the following:
Movies S1 to S4
MDAR Reproducibility Checklist

REFERENCES AND NOTES

1. Y. LeCun, Y. Bengio, Reflections from the turing award winners, in *International Conference on Learning Representations (ICLR)* (2020).
2. B. Neyshabur, Z. Li, S. Bhojanapalli, Y. LeCun, N. Srebro, The role of over-parametrization in generalization of neural networks, in *7th International Conference on Learning Representations, ICLR 2019* (2019).
3. V. Vapnik, *The Nature of Statistical Learning Theory* (Springer science & business media, 1999).
4. S. Schaal, Is imitation learning the route to humanoid robots? *Trends Cogn. Sci.* **3**, 233–242 (1999).
5. S. Ross, G. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (JMLR Workshop and Conference Proceedings, 2011), pp. 627–635.
6. T. Zhang, Z. M. Carthy, O. Jow, D. Lee, X. Chen, K. Goldberg, P. Abbeel, Deep imitation learning for complex manipulation tasks from virtual reality teleoperation, in *2018 IEEE International Conference on Robotics and Automation (ICRA)* (IEEE, 2018), pp. 5628–5635.
7. E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, C. Finn, Bc-z: Zero-shot task generalization with robotic imitation learning, in *Conference on Robot Learning* (PMLR, 2022), pp. 991–1002.
8. P. Goyal, R. J. Mooney, S. Niekum, Zero-shot task adaptation using natural language. arXiv:2106.02972 [cs.AI] (5 June 2021).
9. F. Codevilla, M. Mü ller, A. López, V. Koltun, A. Dosovitskiy, End-to-end driving via conditional imitation learning, in *2018 IEEE International Conference on Robotics and Automation (ICRA)* (IEEE, 2018), pp. 4693–4700.
10. D. Ghosh, A. Gupta, A. Reddy, J. Fu, C. Devin, B. Eysenbach, S. Levine, Learning to reach goals via iterated supervised learning, in *International Conference on Learning Representations* (ICLR, 2020).
11. C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, P. Sermanet, Learning latent plans from play, in *Conference on Robot Learning* (PMLR, 2020), pp. 1113–1132.
12. S. Emmons, B. Eysenbach, I. Kostrikov, S. Levine, Rvs: What is essential for offline rl via supervised learning? arXiv:2112.10751 [cs.LG] (20 December 2021).
13. R. K. Srivastava, P. Shyam, F. Mutz, W. Jaśkowski, J. Schmidhuber, Training agents using upside-down reinforcement learning. arXiv:1912.02877 [cs.LG] (5 December 2019).
14. G. Neumann, J. Peters, Fitted q-iteration by advantage weighted regression, in *Advances in Neural Information Processing Systems* (Curran Associates Inc., 2008) vol. 21.
15. L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, I. Mordatch, Decision transformer: Reinforcement learning via sequence modeling, in *Advances in Neural Information Processing Systems* (2021), vol. 34.
16. S. James, M. Bloesch, A. J. Davison, Task-embedded control networks for few-shot imitation learning, in *Conference on Robot Learning* (PMLR, 2018), pp. 783–795.
17. C. Finn, T. Yu, T. Zhang, P. Abbeel, S. Levine, One-shot visual imitation learning via meta-learning, *Conference on Robot Learning* (PMLR, 2017), pp. 357–368.
18. M. Lechner, R. Hasani, A. Amini, T. A. Henzinger, D. Rus, R. Grosu, Neural circuit policies enabling auditable autonomy. *Nat. Mach. Intell.* **2**, 642–652 (2020).

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

分析方法

VisualBackprop。本文的一个主要目标是分析策略网络对任务的理解。我们通过考察策略的 CNN 如何对输入进行编码并提取特征，对策略网络的任务理解进行了可视化分析。各策略共享相同的 CNN 头部架构，因此不同策略之间 CNN 权重的差异可代表下游架构从环境中学习提取的内容。

我们使用 VisualBackProp 算法，利用对环境的观测来计算模型架构中 CNN 头部的注意力映射。该过程如算法 S4 所示，具体如下：当输入通过 CNN 时，对每个特征图沿通道维度取平均并保存。从最后一层开始，将平均后的特征图反卷积至上一层平均特征图的尺寸。再将反卷积后的特征图与上一层的平均特征图相乘。随后将该乘积传递给上一层。反卷积和乘法被迭代重复，每一步都进行更新，直到到达第一层。结果是一张特征图，显示源图像中模型关注最多的位置。这些图即显著性图（56）。

压力测试。对于图 2 中的压力测试，我们对闭环运行网络记录的 10 次运行的轨迹偏差取平均。其中五次来自训练环境，五次来自城市测试任务。扰动运行数据采集自多种不同的网络类型，包括液态网络和常规网络。

在每次运行中，我们仅输入40%的帧，然后测量最终距离，因为序列越长，轨迹发散得越厉害。由于输入模型的图像是固定的，我们无法对视角发散所导致的透视变化进行建模，任何对未来位置的估计都会变得更不准确。为了限制离线帧生成的影响，同时仍保留足够的帧以产生具有统计显著性的结果，我们选择只保留每个序列的前40%。

基于注意力的徒步任务设置。我们设计了一个控制器，它依赖每个预训练网络架构的卷积层所感知到的注意力图，来触发两个目标之间的跳转。我们观察到，对于对任务有一定理解的网络，当感兴趣的目标足够近时，显著性图会显示出与输入图像中物体位置相对应的明亮区域。因此，我们使用面积占用判据来触发目标切换。当网络正在操控四旋翼飞行器时，我们分析每一帧显著性图，将低于亮度阈值的像素掩蔽，并检测最显著且最大的连通区域。当最佳候选区域的像素面积超过固定阈值（对应无人机距离露营椅 2 m）时，我们认为已到达目标，并触发覆盖网络指令，用固定时长的油门指令使四旋翼飞行器上升 1.6 m。这确保了当前目标离开视野，并且网络能够获取场景下一部分的图像。控制权交还给网络，直到检测到新目标，如此循环。这种控制协议（如图 S3 所示）在设计上可以处理任意数量的检查点，前提是网络能够看到并能够导航到下一个目标。

统计分析

论文的定量结果在表 1 和表 2 以及表 S4 至 S7 中给出，均提供了所执行的主动测试数量 N，并以百分比报告了模型在各场景中的成功率。在图 2 的所有子图中，每个点表示 10 次运行的平均值，而每个点上的竖线表示与该数据条目对应的标准差。

补充材料

本PDF文件包括：材料与方法、图S1至S10、表S1至S7、参考文献（57–61）

本手稿的其他补充材料包括以下内容：

视频S1至S4 MDAR可重复性检查表

参考文献与注释。

1. Y. LeCun, Y. Bengio, 《图灵奖得主的反思》，载于《国际学习表征会议》（ICLR，2020）。2. B. Neyshabur，Z. Li，S. Bhojanapalli，Y. LeCun，N. Srebro，过参数化在神经网络泛化中的作用，载于《第七届国际学习表征会议》（ICLR 2019，2019）。3. V. Vapnik，《统计学习理论的本质》（Springer Science & Business Media，1999）。4. S. Schaal，模仿学习是通往人形机器人的途径吗？《认知科学趋势》3，233–242（1999）。5. S. Ross，G. Gordon，D. Bagnell，将模仿学习与结构化预测归约为无悔在线学习，载于《第十四届人工智能与统计国际会议论文集》（JMLR Workshop and Conference Proceedings，2011），第627–635页。6. T. Zhang，Z. M. Carthy，O. Jow，D. Lee，X. Chen，K. Goldberg，P. Abbeel，基于虚拟现实遥操作的复杂操作任务深度模仿学习，载于《2018 IEEE 国际机器人与自动化会议》（ICRA）（IEEE，2018），第5628–5635页。7. E. Jang，A. Irpan，M. Khansari，D. Kappler，F. Ebert，C. Lynch，S. Levine，C. Finn，BC-Z：利用机器人模仿学习实现零样本任务泛化，载于《机器人学习会议》（PMLR，2022），第991–1002页。8. P. Goyal，R. J. Mooney，S. Niekum，使用自然语言进行零样本任务适应。arXiv:2106.02972 [cs.AI]（2021年6月5日）。9. F. Codevilla，M. Mü ller，A. López，V. Koltun，A. Dosovitskiy，通过条件模仿学习进行端到端驾驶，载于《2018 IEEE 国际机器人与自动化会议》（ICRA）（IEEE，2018），第4693–4700页。10. D. Ghosh，A. Gupta，A. Reddy，J. Fu，C. Devin，B. Eysenbach，S. Levine，通过迭代监督学习学习达成目标，载于《国际学习表征会议》（ICLR，2020）。11. C. Lynch，M. Khansari，T. Xiao，V. Kumar，J. Tompson，S. Levine，P. Sermanet，从玩耍中学习潜在规划，载于《机器人学习会议》（PMLR，2020），第1113–1132页。12. S. Emmons，B. Eysenbach，I. Kostrikov，S. Levine，RVS：通过监督学习进行离线强化学习的关键是什么？arXiv:2112.10751 [cs.LG]（2021年12月20日）。13. R. K. Srivastava，P. Shyam，F. Mutz，W. Jaśkowski，J. Schmidhuber，使用倒置强化学习训练智能体。arXiv:1912.02877 [cs.LG]（2019年12月5日）。14. G. Neumann，J. Peters，通过优势加权回归进行拟合Q迭代，载于《神经信息处理系统进展》（Curran Associates Inc.，2008），第21卷。15. L. Chen，K. Lu，A. Rajeswaran，K. Lee，A. Grover，M. Laskin，P. Abbeel，A. Srinivas，I. Mordatch，决策Transformer：通过序列建模进行强化学习，载于《神经信息处理系统进展》（2021），第34卷。16. S. James，M. Bloesch，A. J. Davison，用于小样本模仿学习的任务嵌入控制网络，载于《机器人学习会议》（PMLR，2018），第783–795页。17. C. Finn，T. Yu，T. Zhang，P. Abbeel，S. Levine，通过元学习进行单样本视觉模仿学习，《机器人学习会议》（PMLR，2017），第357–368页。18. M. Lechner，R. Hasani，A. Amini，T. A. Henzinger，D. Rus，R. Grosu，实现可审计自主性的神经回路策略。《自然·机器智能》2，642–652（2020）。

19. M. Lechner, R. Hasani, M. Zimmer, T. A. Henzinger, R. Grosu, Designing worm inspired neural networks for interpretable robotic control, in *2019 International Conference on Robotics and Automation (ICRA)* (IEEE, 2019), pp. 87–94.
20. R. Hasani, M. Lechner, A. Amini, D. Rus, R. Grosu, A natural lottery ticket winner: Reinforcement learning with ordinary neural circuits, in *International Conference on Machine Learning* (PMLR, 2020), pp. 4082–4093.
21. C. Vorbach, R. Hasani, A. Amini, M. Lechner, D. Rus, Causal navigation by continuous-time neural networks, in *Advances in Neural Information Processing Systems* (2021), vol. 34.
22. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in *Advances in Neural Information Processing Systems* (NIPS, 2017), pp. 5998–6008.
23. S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg, T. Eccles, J. Bruce, A. Razavi, A. Edwards, N. Heess, Y. Chen, R. Hadsell, O. Vinyals, M. Bordbar, N. de Freitas, A generalist agent. arXiv:2205.06175 [cs.AI] (12 May 2022).
24. J. Kaplan, S. M. Candlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models. arXiv:2001.08361 [cs.LG] (23 January 2020).
25. S. Bubeck, M. Sellke, A universal law of robustness via isoperimetry, in *Advances in Neural Information Processing Systems* (NeurIPS, 2021), vol. 34.
26. J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, Diego de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, L. Sifre, Training compute-optimal large language models. arXiv:2203.15556 [cs.CL] (29 March 2022).
27. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. M. Candlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in *Advances in Neural Information Processing Systems* (NeurIPS, 2020), vol. 33, p. 1877.
28. V. Sanh, A. Webson, C. Raffel, S. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, A. Raja, M. Dey, M Saiful Bari, C. Xu, U. Thakker, S. S. Sharma, E. Szczechla, T. Kim, G. Chhablani, N. Nayak, D. Datta, J. Chang, Mike Tian-Jian Jiang, H. Wang, M. Manica, S. Shen, Z. X. Yong, H. Pandey, R. Bawden, T. Wang, T. Neeraj, J. Rozen, A. Sharma, A. Santilli, T. Fevry, J. A. Fries, R. Teehan, T. L. Scao, S. Biderman, L. Gao, T. Wolf, A. M. Rush, Multitask prompted training enables zero-shot task generalization, in *The Tenth International Conference on Learning Representations* (ICLR, 2022).
29. J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, Q. V. Le, Finetuned language models are zero-shot learners. arXiv:2109.01652 [cs.CL] (3 Sep 2021).
30. C. Jia, Y. Yang, Y. Xia, Y.-T. Chen, Z. Parekh, H. Pham, Q. V. Le, Y. Sung, Z. Li, T. Duerig, Scaling up visual and vision-language representation learning with noisy text supervision, in *International Conference on Machine Learning* (PMLR, 2021), pp. 4904–4916.
31. J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, K. Simonyan, Flamingo: A visual language model for few-shot learning. arXiv:2204.14198 [cs.CV] (29 April 2022).
32. A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, M. Chen, Hierarchical text-conditional image generation with clip latents. arXiv:2204.06125 [cs.CV] (13 April 2022).
33. J. Zhang, K. Yang, C. Ma, S. Reiß, K. Peng, R. Stiefelhagen, Bending reality: Distortion-aware transformers for adapting to panoramic semantic segmentation, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, 2022), pp. 16917–16927.
34. A. Prakash, K. Chitta, A. Geiger, Multi-modal fusion transformer for end-to-end autonomous driving, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, 2021), pp. 7077–7087.
35. R. Hasani, M. Lechner, A. Amini, D. Rus, R. Grosu, Liquid time constant networks, in *Proceedings of the AAAI Conference on Artificial Intelligence* (AAAI, 2021), vol. 35, p. 7657.
36. B. Schölkopf, Causality for machine learning. arXiv:1911.10500 [cs.LG] (24 Nov 2019).
37. S. Hochreiter, J. Schmidhuber, Long short-term memory. *Neural Comput.* **9**, 1735–1780 (1997).
38. Y. Rubanova, R. T. Chen, D. K. Duvenaud, Latent ordinary differential equations for irregularly-sampled time series, in *Advances in Neural Information Processing Systems* (NeurIPS, 2019), vol. 32.
39. K. Cho, B. Van Merriënboer, D. Bahdanau, Y. Bengio, On the properties of neural machine translation: Encoder-decoder approaches. arXiv:1409.1259 (3 Sep 2014).
40. C. Lea, R. Vidal, A. Reiter, G. D. Hager, Temporal convolutional networks: A unified approach to action segmentation, in *European Conference on Computer Vision* (Springer, 2016), pp. 47–54.
41. R. T. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural ordinary differential equations, in *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (NeurIPS, 2018), pp. 6572–6583.
42. S. Bai, J. Z. Kolter, V. Koltun, An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271 [cs.LG] (4 Mar 2018).
43. K. J. Friston, L. Harrison, W. Penny, Dynamic causal modelling. *Neuroimage* **19**, 1273–1302 (2003).
44. M. Bojarski, A. Choromanska, K. Choromanski, B. Firner, L. J. Ackel, U. Muller, P. Yeres, K. Zieba, Visualbackprop: Efficient visualization of cnns for autonomous driving, in *IEEE International Conference on Robotics and Automation (ICRA)* (ICRA, 2018), pp. 1–8.
45. J. Pearl, Causal inference in statistics: An overview. *Stat. Surv.* **3**, 96–146 (2009).
46. W. Penny, Z. Ghahramani, K. Friston, Bilinear dynamical systems. *Phil. Trans. R. Soc. B Biol. Sci.* **360**, 983–993 (2005).
47. C. Koch, I. Segev, *Methods in Neuronal Modeling: From Ions to Networks* (MIT Press, 1998).
48. E. R. Kandel, J. H. Schwartz, T. M. Jessell, *Principles of Neural Science* (McGraw-Hill, 2000), vol. 4.
49. L. Lapique, Recherches quantitatives sur l'excitation electrique des nerfs traitee comme une polarization. *J. Physiol. Pathol.* **9**, 620 (1907).
50. R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Teschl, D. Rus, Closed-form continuous-time neural models. arXiv:2106.13898 [cs.LG] (25 June 2021).
51. G. P. Sarma, C. W. Lee, T. Portegys, V. Ghayoomie, T. Jacobs, B. Alicea, M. Cantarelli, M. Currie, R. C. Gerkin, S. Gingell, P. Gleeson, R. Gordon, R. M. Hasani, G. Idili, S. Khayrulin, D. Lung, A. Palyanov, M. Watts, S. D. Larson, OpenWorm: Overview and recent advances in integrative biological simulation of *Caenorhabditis elegans*. *Philos. Trans. R. Soc. B* **373**, 20170382 (2018).
52. K. Friston, Causal modelling and brain connectivity in functional magnetic resonance imaging. *PLOS Biol.* **7**, e1000033 (2009).
53. J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, in *Advances in Neural Information Processing Systems* (NIPS, 2011), vol. 24.
54. J. Bergstra, D. Yamins, D. Cox, Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures, in *International Conference on Machine Learning* (PMLR, 2013), pp. 115–123.
55. Stanford Artificial Intelligence Laboratory *et al.*, Robotic operating system.
56. M. Bojarski, A. Choromanska, K. Choromanski, B. Firner, L. Jackel, U. Muller, K. Zieba, Visualbackprop: efficient visualization of CNNs. arXiv:1611.05418 [cs.CV] (16 November 2017).
57. S. Ross, D. Bagnell, Efficient reductions for imitation learning, in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (JMLR Workshop and Conference Proceedings, 2010), pp. 661–668.
58. R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction* (MIT press, 2018).
59. J. Chen, B. Yuan, M. Tomizuka, Deep imitation learning for autonomous driving in generic urban scenarios with enhanced safety, in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (IEEE, 2019), pp. 2884–2890.
60. X. Li, P. Ye, J. Jin, F. Zhu, F.-Y. Wang, Data augmented deep behavioral cloning for urban traffic control operations under a parallel learning framework, in *IEEE Transactions on Intelligent Transportation Systems* (IEEE, 2021).
61. A. Amini, T.-H. Wang, I. Gilitschenski, W. Schwarting, Z. Liu, S. Han, S. Karaman, D. Rus, Vista 2.0: An open, data-driven simulator for multimodal sensing and policy learning for autonomous vehicles. arXiv:2111.12083 [cs.RO] (23 November 2021).

Acknowledgments: We would like to thank J. Feld, I. Futran, K. Mishra, P. Whartenby, C. Vorbach, T.-H. Wang, and W. Xiao for testing NCP and CfC models for confirmation of results across other domains. We also thank S. T. Monteiro of Aurora Flight Sciences for providing constructive feedback and validating our experimental setting throughout this project. **Funding:** This research was supported in part by the AI2050 program at Schmidt Futures (grant G22-63172), the United States Air Force Research Laboratory, and the United States Air Force Artificial Intelligence Accelerator and was accomplished under Cooperative Agreement Number FA8750-19-2-1000. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Air Force or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes, notwithstanding any copyright notation herein. This work was further partially supported by the Boeing Company. A.R. gratefully acknowledges support from a National Defense Science and Engineering Graduate Fellowship. **Author contributions:** R.H., A.R., P.K., M.C., and D.R. conceptualized, designed, performed research, and analyzed data. R.S., A.A., and M.L. contributed to designing research, data curation, research implementation, and analyzing data. D.R. helped with the design of the research and guided and supervised the work. All authors wrote the article. **Competing interests:** R.H. is also affiliated with the Vanguard Group. R.S. is currently affiliated with Altitude AI. P.K. is currently affiliated with Wing. D.R. is affiliated with the Mitre Corporation as a senior visiting fellow. This work in part has been filed as a provisional

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

19. M. Lechner，R. Hasani，M. Zimmer，T. A. Henzinger，R. Grosu，设计受线虫启发的神经网络用于可解释机器人控制，载于《2019 国际机器人与自动化会议》（ICRA）（IEEE，2019），第87–94页。20. R. Hasani，M. Lechner，A. Amini，D. Rus，R. Grosu，自然彩票中奖者：使用普通神经回路进行强化学习，载于《国际机器学习会议》（PMLR，2020），第4082–4093页。21. C. Vorbach，R. Hasani，A. Amini，M. Lechner，D. Rus，通过连续时间神经网络进行因果导航，载于《神经信息处理系统进展》（2021），第34卷。22. A. Vaswani，N. Shazeer，N. Parmar，J. Uszkoreit，L. Jones，A. N. Gomez，L. Kaiser，I. Polosukhin，注意力就是你所需要的一切，载于《神经信息处理系统进展》（NIPS，2017），第5998–6008页。23. S. Reed，K. Zolna，E. Parisotto，S. G. Colmenarejo，A. Novikov，G. Barth-Maron，M. Gimenez，Y. Sulsky，J. Kay，J. T. Springenberg，T. Eccles，J. Bruce，A. Razavi，A. Edwards，N. Heess，Y. Chen，R. Hadsell，O. Vinyals，M. Bordbar，N. de Freitas，通用智能体。arXiv:2205.06175 [cs.AI]（2022年5月12日）。24. J. Kaplan，S. M. Candlish，T. Henighan，T. B. Brown，B. Chess，R. Child，S. Gray，A. Radford，J. Wu，D. Amodei，神经语言模型的缩放定律。arXiv:2001.08361 [cs.LG]（2020年1月23日）。25. S. Bubeck，M. Sellke，通过等周不等式得到的鲁棒性普适定律，载于《神经信息处理系统进展》（NeurIPS，2021），第34卷。26. J. Hoffmann，S. Borgeaud，A. Mensch，E. Buchatskaya，T. Cai，E. Rutherford，Diego de Las Casas，L. A. Hendricks，J. Welbl，A. Clark，T. Hennigan，E. Noland，K. Millican，G. van den Driessche，B. Damoc，A. Guy，S. Osindero，K. Simonyan，E. Elsen，J. W. Rae，O. Vinyals，L. Sifre，训练计算最优的大语言模型。arXiv:2203.15556 [cs.CL]（2022年3月29日）。27. T. Brown，B. Mann，N. Ryder，M. Subbiah，J. D. Kaplan，P. Dhariwal，A. Neelakantan，P. Shyam，G. Sastry，A. Askell，S.Agarwal，A. Herbert-Voss，G. Krueger，T. Henighan，R. Child，A. Ramesh，D. Ziegler，J. Wu，C. Winter，C. Hesse，M. Chen，E. Sigler，M. Litwin，S. Gray，B. Chess，J. Clark，C. Berner，S. M. Candlish，A. Radford，I. Sutskever，D. Amodei，语言模型是小样本学习器，载于《神经信息处理系统进展》（NeurIPS，2020），第33卷，第1877页。28. V. Sanh，A. Webson，C. Raffel，S. Bach，L. Sutawika，Z. Alyafeai，A. Chaffin，A. Stiegler，A. Raja，M. Dey，M Saiful Bari，C. Xu，U. Thakker，S. S. Sharma，E. Szczechla，T. Kim，G. Chhablani，N. Nayak，D. Datta，J. Chang，Mike Tian-Jian Jiang，H. Wang，M. Manica，S. Shen，Z. X. Yong，H. Pandey，R. Bawden，T. Wang，T. Neeraj，J. Rozen，A. Sharma，A. Santilli，T. Fevry，J. A. Fries，R. Teehan，T. L. Scao，S. Biderman，L. Gao，T. Wolf，A. M. Rush，多任务提示训练实现零样本任务泛化，载于《第十届国际学习表征会议》（ICLR，2022）。29. J. Wei，M. Bosma，V. Y. Zhao，K. Guu，A. W. Yu，B. Lester，N. Du，A. M. Dai，Q. V. Le，微调后的语言模型是零样本学习器。arXiv:2109.01652 [cs.CL]（2021年9月3日）。30. C. Jia，Y. Yang，Y. Xia，Y.-T. Chen，Z. Parekh，H. Pham，Q. V. Le，Y. Sung，Z. Li，T. Duerig，利用噪声文本监督扩展视觉与视觉-语言表征学习，载于《国际机器学习会议》（PMLR，2021），第4904–4916页。31. J.-B. Alayrac，J. Donahue，P. Luc，A. Miech，I. Barr，Y. Hasson，K. Lenc，A. Mensch，K. Millican，M. Reynolds，R. Ring，E. Rutherford，S. Cabi，T. Han，Z. Gong，S. Samangooei，M. Monteiro，J. Menick，S. Borgeaud，A. Brock，A. Nematzadeh，S. Sharifzadeh，M. Binkowski，R. Barreira，O. Vinyals，A. Zisserman，K. Simonyan，Flamingo：用于小样本学习的视觉语言模型。arXiv:2204.14198 [cs.CV]（2022年4月29日）。32. A. Ramesh，P. Dhariwal，A. Nichol，C. Chu，M. Chen，使用CLIP潜变量的分层文本条件图像生成。arXiv:2204.06125 [cs.CV]（2022年4月13日）。33. J. Zhang，K. Yang，C. Ma，S. Reiß，K. Peng，R. Stiefelhagen，弯曲现实：用于适应全景语义分割的失真感知Transformer，载于《IEEE/CVF 计算机视觉与模式识别会议论文集》（IEEE，2022），第16917–16927页。34. A. Prakash，K. Chitta，A. Geiger，用于端到端自动驾驶的多模态融合Transformer，载于《IEEE/CVF 计算机视觉与模式识别会议论文集》（IEEE，2021），第7077–7087页。35. R. Hasani，M. Lechner，A. Amini，D. Rus，R. Grosu，液态时间常数网络，载于《AAAI 人工智能会议论文集》（AAAI，2021），第35卷，第7657页。36. B. Schölkopf，机器学习的因果性。arXiv:1911.10500 [cs.LG]（2019年11月24日）。37. S. Hochreiter，J. Schmidhuber，长短期记忆。《神经计算》9，1735–1780（1997）。38. Y. Rubanova，R. T. Chen，D. K. Duvenaud，用于不规则采样时间序列的潜在常微分方程，载于《神经信息处理系统进展》（NeurIPS，2019），第32卷。39. K. Cho，B. Van Merriënboer，D. Bahdanau，Y. Bengio，论神经机器翻译的性质：编码器-解码器方法。arXiv:1409.1259（2014年9月3日）。40. C. Lea，R. Vidal，A. Reiter，G. D. Hager，时间卷积网络：动作分割的统一方法，载于《欧洲计算机视觉会议》（Springer，2016），第47–54页。
41. R. T. Chen，Y. Rubanova，J. Bettencourt，D. Duvenaud，神经常微分方程，载于第32届国际神经信息处理系统会议论文集（NeurIPS，2018），第6572–6583页。42. S. Bai，J. Z. Kolter，V. Koltun，对用于序列建模的通用卷积网络和循环网络的经验评估。arXiv:1803.01271 [cs.LG]（2018年3月4日）。43. K. J. Friston，L. Harrison，W. Penny，动态因果建模。Neuroimage 19，1273–1302（2003）。44. M. Bojarski，A. Choromanska，K. Choromanski，B. Firner，L. J. Ackel，U. Muller，P. Yeres，K. Zieba，Visualbackprop：面向自动驾驶的CNN高效可视化，载于IEEE机器人与自动化国际会议（ICRA）（ICRA，2018），第1–8页。45. J. Pearl，统计学中的因果推断：概述。Stat. Surv. 3，96–146（2009）。46. W. Penny，Z. Ghahramani，K. Friston，双线性动力系统。Phil. Trans. R. Soc. B Biol. Sci. 360，983–993（2005）。47. C. Koch，I. Segev，《神经元建模方法：从离子到网络》（MIT Press，1998）。48. E. R. Kandel，J. H. Schwartz，T. M. Jessell，《神经科学原理》（McGraw-Hill，2000），第4卷。49. L. Lapique，关于将神经电兴奋视为极化的定量研究。J. Physiol. Pathol. 9，620（1907）。50. R. Hasani，M. Lechner，A. Amini，L. Liebenwein，A. Ray，M. Tschaikowski，G. Teschl，D. Rus，闭式连续时间神经模型。arXiv:2106.13898 [cs.LG]（2021年6月25日）。51. G. P. Sarma，C. W. Lee，T. Portegys，V. Ghayoomie，T. Jacobs，B. Alicea，M. Cantarelli，M. Currie，R. C. Gerkin，S. Gingell，P. Gleeson，R. Gordon，R. M. Hasani，G. Idili，S. Khayrulin，D. Lung，A. Palyanov，M. Watts，S. D. Larson，OpenWorm：秀丽隐杆线虫整合生物学模拟的概述与最新进展。Philos. Trans. R. Soc. B 373，20170382（2018）。52. K. Friston，功能磁共振成像中的因果建模与脑连接。PLOS Biol. 7，e1000033（2009）。53. J. Bergstra，R. Bardenet，Y. Bengio，B. Kégl，超参数优化算法，载于《神经信息处理系统进展》（NIPS，2011），第24卷。54. J. Bergstra，D. Yamins，D. Cox，让模型搜索成为科学：面向视觉架构的数百维超参数优化，载于国际机器学习会议（PMLR，2013），第115–123页。55. 斯坦福人工智能实验室等，机器人操作系统。56. M. Bojarski，A. Choromanska，K. Choromanski，B. Firner，L. Jackel，U. Muller，K. Zieba，Visualbackprop：CNN的高效可视化。arXiv:1611.05418 [cs.CV]（2017年11月16日）。57. S. Ross，D. Bagnell，模仿学习的高效约简，载于第十三届人工智能与统计国际会议论文集（JMLR工作坊与会议论文集，2010），第661–668页。58. R. S. Sutton，A. G. Barto，《强化学习：导论》（MIT Press，2018）。59. J. Chen，B. Yuan，M. Tomizuka，面向具有增强安全性的通用城市场景中自动驾驶的深度模仿学习，载于2019年IEEE/RSJ智能机器人与系统国际会议（IROS）（IEEE，2019），第2884–2890页。60. X. Li，P. Ye，J. Jin，F. Zhu，F.-Y. Wang，平行学习框架下用于城市交通控制操作的数据增强深度行为克隆，载于IEEE智能交通系统汇刊（IEEE，2021）。61. A. Amini，T.-H. Wang，I. Gilitschenski，W. Schwarting，Z. Liu，S. Han，S. Karaman，D. Rus，Vista 2.0：面向自动驾驶车辆的多模态感知与策略学习的开放、数据驱动模拟器。arXiv:2111.12083 [cs.RO]（2021年11月23日）。

致谢：我们感谢 J. Feld、I. Futran、K. Mishra、P. Whartenby、C. Vorbach、T.-H. Wang 和 W. Xiao 测试 NCP 和 CfC 模型，以在其他领域确认结果。我们还感谢 Aurora Flight Sciences 的 S. T. Monteiro 在整个项目中提供建设性反馈并验证我们的实验设置。资助：本研究部分得到 Schmidt Futures 的 AI2050 计划（资助编号 G22-63172）、美国空军研究实验室以及美国空军人工智能加速器的支持，并在合作协议编号 FA8750-19-2-1000 下完成。本文档所载观点和结论仅代表作者本人，不应被解释为美国空军或美国政府的官方政策，无论明示或暗示。美国政府有权出于政府目的复制和分发重印本，尽管本文载有任何版权声明。本工作还部分得到波音公司的支持。A.R. 谨对国防科学与工程研究生奖学金的支持表示感谢。作者贡献：R.H.、A.R.、P.K.、M.C. 和 D.R. 构思、设计、开展研究并分析数据。R.S.、A.A. 和 M.L. 参与研究设计、数据整理、研究实施和数据分析。D.R. 协助研究设计，并指导与监督本工作。所有作者撰写了本文。利益冲突：R.H. 还隶属于 Vanguard Group。R.S. 目前隶属于 Altitude AI。P.K. 目前隶属于 Wing。D.R. 以高级访问研究员身份隶属于 Mitre Corporation。本工作已部分作为一项临时

patent with application number 63/415,382 on 12 October 2022. The other authors declare that they have no competing interests. **Data and materials availability:** All data, code, and materials used in the analysis are openly available at https://zenodo.org/badge/latestdoi/610810400 and https://zenodo.org/badge/latestdoi/381393816 under Apache 2.0 License for purposes of reproducing and extending the analysis. The original hand-collected training dataset is published (http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN), with a file name of devens snowy and a fixed size of 33.2 GB. This dataset was used for training the starting checkpoints. The dataset used for fine-tuning the models tested on the drone can be found at http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN (devens chair, 2.3 GB) and has a subset of the full dataset containing only runs with the chair target. We have also included the exact synthetic datasets we used for our experiments. We have both a full dataset http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN (synthetic small4, 14.7 GB) used to train the starting checkpoint and a synthetic chair–only dataset http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN (synthetic chair, 4.3 GB) used to fine-tune the final models for online testing. We would like to thank IBM for providing access to the Satori computing cluster and MIT/MIT Lincoln Labs for providing access to the Supercloud computing cluster. Both clusters were very helpful for training models and hyperparameter optimization.

Submitted 6 May 2022
Accepted 22 March 2023
Published 19 April 2023
10.1126/scirobotics.adc8892

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

专利，申请号为 63/415,382，于 2022 年 10 月 12 日提交。其他作者声明无利益冲突。数据与材料可用性：分析中使用的所有数据、代码和材料均可在 https://zenodo.org/badge/latestdoi/610810400 和 https://zenodo.org/badge/latestdoi/381393816 公开获取；依据 Apache 2.0 许可证，可用于复现和扩展该分析。原始手工收集的训练数据集已发布（http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN& fid=06lMJMN），文件名为 devens snowy，固定大小为 33.2 GB。该数据集用于训练初始检查点。用于微调在无人机上测试的模型的数据集可在 http://knightrider mit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN 找到（devens chair，2.3 GB），它包含完整数据集的一个子集，仅含以椅子为目标对象的运行记录。我们还附上了实验中所使用的确切合成数据集。我们同时提供了完整数据集 http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN（合成 small4，14.7 GB）用于训练初始检查点，以及一个仅包含椅子的合成数据集 http://knightridermit.myqnapcloud.com:8080/share.cgi?ssid=06lMJMN&fid=06lMJMN（合成椅子，4.3 GB）用于为在线测试微调最终模型。我们要感谢 IBM 提供对 Satori 计算集群的访问权限，并感谢 MIT/MIT Lincoln Labs 提供对 Supercloud 计算集群的访问权限。这两个集群对训练模型和超参数优化都非常有帮助。

2022年5月6日提交 2023年3月
22日接受 2023年4月19日发表
1
0.1126/scirobotics.adc8892

Science Robotics

Robust flight navigation out of distribution with liquid neural networks

Makram Chahine, Ramin Hasani, Patrick Kao, Aaron Ray, Ryan Shubert, Mathias Lechner, Alexander Amini, and Daniela Rus

Sci. Robot. **8** (77), eadc8892. DOI: 10.1126/scirobotics.adc8892

View the article online
https://www.science.org/doi/10.1126/scirobotics.adc8892
Permissions
https://www.science.org/help/reprints-and-permissions

Downloaded from https://www.science.org at National University of Defense Technology on November 03, 2026

Use of this article is subject to the Terms of service

Science Robotics (ISSN 2470-9476) is published by the American Association for the Advancement of Science. 1200 New York Avenue NW, Washington, DC 20005. The title *Science Robotics* is a registered trademark of AAAS.

Copyright © 2023 The Authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original U.S. Government Works

Science Robotics

基于液体神经网络的分布外鲁棒飞行导航

Makram Chahine、Ramin Hasani、Patrick Kao、Aaron Ray、Ryan Shubert、Mathias Lechner、Alexander Amini 和 Daniela Rus

科学机器人 8 (77), eadc8892. DOI: 10.1126/scirobotics.adc8892

在线查看文章
https://www.science.org/doi/10.1126/scirobotics.adc8892 许可 https://www.science.org/help/reprints-and-permissions

本文的使用须遵守服务条款

Science Robotics（ISSN 2470-9476）由美国科学促进会出版。1200 New York Avenue NW, Washington, DC 20005。Science Robotics 这一名称是 AAAS 的注册商标。

版权所有 © 2023 作者，保留部分权利；美国科学促进会为独家被许可方。不对美国政府原创作品主张权利